M03 — Prácticas de Blazor Web App y componentes

Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

CURSO: Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: PRÁCTICAS

AUTOR: JAIME GALLO

Itinerario de trabajo y condiciones del laboratorio

Durante esta jornada utilizaremos una única aplicación B de Blazor Web App en .NET 8, organizada en seis **estados reproducibles** dentro de la carpeta `M03_BLAZOR/PROYECTO/`. Cada punto tiene un proyecto `CursoWeb.Blazor` independiente que conserva la misma estructura general. Eso permite repetir un experimento desde su estado inicial sin desmontar el código de prácticas anteriores; no representa seis aplicaciones de negocio distintas. El alumno conoce C# y las bases de Web API, pero no se presupone que conozca Blazor. No se requiere SQL Server, LocalDB ni API remota para M03.

Los proyectos están preparados con comentarios pedagógicos identificados por paso. En archivos C#, se activa el bloque delimitado por `/* PASO X INICIO` y `PASO X FIN */` retirando **únicamente los delimitadores**. En archivos Razor se retiran `@*` y `*@` del fragmento correspondiente. Cuando se proponen variantes completas de un componente, hay que **sustituir** una por otra, no dejar varias instancias visibles simultáneamente. El alumno no tiene que adivinar qué líneas aparecieron entre un estado y el siguiente.

Antes de cada práctica detén el proceso de la anterior y abre el `CursoWeb.Blazor.csproj` de la carpeta concreta. Para P3.1, por ejemplo:

```
cd M03_BLAZOR/PROYECTO/3.1/CursoWeb.Blazor
dotnet restore
dotnet build
dotnet run --no-launch-profile --urls http://localhost:5092
```

Abre después `http://localhost:5092`. Si Visual Studio utiliza un puerto distinto, sigue la dirección anunciada por Kestrel. El proyecto debe ejecutarse con el SDK de .NET 8. No copies los cambios en M01 ni M02; son aplicaciones y materiales anteriores. Los pasos esenciales forman la práctica de clase; los experimentos de transferencia y retos amplían la comprensión sin requerir una infraestructura adicional.

Índice del cuaderno

- P3.1. Comparación de SSR estático e Interactive Server.
- P3.2. Página Productos y navegación con NavLink.
- P3.3. Parámetros y componente TarjetaProducto.
- P3.4. Estado, eventos y binding.
- P3.5. Comunicación padre–hijo con EventCallback.
- P3.6. Servicio asíncrono, OnInitializedAsync y estados de carga.

P3.1. Crear la aplicación Blazor B y comprobar interactividad real

Objetivo y estado inicial

El objetivo es identificar el problema que resuelve Interactive Server y comprobarlo empíricamente. La carpeta inicial es `M03_BLAZOR/PROYECTO/3.1/CursoWeb.Blazor`. Dispone de dos páginas equivalentes en lo esencial: `Estatica.razor` sin modo interactivo y `Interactiva.razor` con ese modo declarado. El primero permite observar que escribir un manejador de clic no es suficiente; el segundo comprueba que el mismo tipo de manejador sí funciona cuando la aplicación está configurada correctamente.

3.1.1. Abrir y compilar el estado de partida

Abre exactamente `M03_BLAZOR/PROYECTO/3.1/CursoWeb.Blazor/CursoWeb.Blazor.csproj`. Comprueba que el archivo de proyecto declara `net8.0`. En una consola situada en esa carpeta, ejecuta `dotnet restore` y `dotnet build`. Si alguna operación falla, verifica la versión del SDK y que abriste este proyecto, no uno de los módulos MVC.

Ejecuta con `dotnet run --no-launch-profile --urls http://localhost:5092`. El resultado esperado es que Kestrel anuncie la URL. Abre el navegador y visita la raíz. Debes encontrar el título «Aplicación B · Blazor Web App», una descripción del catálogo en memoria y enlaces a las dos formas de renderizado.

Relación con la teoría: 3.1 explica por qué Blazor sigue ejecutándose dentro del host ASP.NET Core aunque cambie el modelo de interacción. En `Program.cs` verifica las dos instrucciones que preparan el servidor:

```
builder.Services.AddRazorComponents().AddInteractiveServerComponents();
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
```

La primera configura servicios; la segunda expone endpoints de componentes con soporte de interactividad en servidor. Si falta una parte, no debemos concluir que un método `@onclick` funcionará solo porque el proyecto contiene código C#.

3.1.2. Identificar las piezas del proyecto

Abre `Components/App.razor` y localiza el documento HTML, `<Routes />` y el script `_framework/blazor.web.js`. Abre `Components/Routes.razor` y encuentra `Router` y `RouteView`. Abre `Components/Layout/MainLayout.razor` y localiza `@Body`. Comprueba que `Components/Pages/Home.razor` declara `@page "/"`.

Explica en tu cuaderno el recorrido de la ruta inicial: el navegador solicita una URL; ASP.NET Core atiende el endpoint; el router identifica la página; el layout la representa dentro de una estructura común. Esta explicación se parece conceptualmente al recorrido MVC que ya conoces, pero ahora no existe `HomeController` para esta pantalla.

Como caso negativo, abre `/una-ruta-inexistente`. Debes ver el contenido de la rama `NotFound` del router. La aplicación debe continuar respondiendo en `/`. Esta observación ayuda a distinguir una ruta inexistente de una aplicación que no ha arrancado.

3.1.3. Observar un botón bajo SSR estático

Abre `Components/Pages/Estatica.razor`. Comprueba que existe:

```
@page "/estatica"
<p id="contador-estatico">Pulsaciones: @contador</p>
<button id="boton-estatico" @onclick="Incrementar">Pulsar (sin interacción)</
button>

@code {
    private int contador;
    private void Incrementar() => contador++;
}
```

Visita `http://localhost:5092/estatica` y pulsa el botón tres veces. **Resultado esperado:** «Pulsaciones: 0» continúa visible. Esto no constituye un fallo del ejercicio. El componente tiene código y se ha representado desde el servidor, pero no está habilitado como componente interactivo capaz de procesar ese clic C# después de la respuesta inicial.

No intentes solucionar este resultado sustituyendo el método por JavaScript. El propósito es aislar una condición del modelo de ejecución. En las herramientas

de desarrollo observa que pulsar no desencadena una navegación completa con una nueva acción MVC, y describe qué necesitaría el componente para responder.

3.1.4. Observar el mismo evento con Interactive Server

Abre `Components/Pages/Interactiva.razor`. Sus primeras líneas son:

```
@page "/interactiva"
@rendermode InteractiveServer
```

El resto conserva un contador y un botón con manejador de clic. Abre `http://localhost:5092/interactiva`, deja que la conexión se establezca y pulsa una vez. Debes ver «Pulsaciones: 1». Vuelve a pulsar: el número pasa a dos. No se modifica el servidor SQL ni una colección global: el campo `contador` pertenece a la instancia interactiva del componente.

Compara qué cambia en los dos archivos: no necesitas escribir otro lenguaje de programación; el elemento diferenciador es `@rendermode InteractiveServer` y el soporte configurado en `Program.cs`. Revisa también que en `App.razor` el proyecto utiliza `<Routes />` sin hacer interactivo todo el árbol; por eso ambas páginas pueden coexistir con comportamientos distintos.

3.1.5. Diagnóstico y verificación negativa

Retira temporalmente la directiva `@rendermode InteractiveServer` de `Interactiva.razor`, recompila y abre esa página desde cero. El botón deja de incrementar el valor, como en `/estatica`. Restablece la directiva y verifica que vuelve a funcionar. En una explicación de una o dos frases, relaciona ambos resultados con la diferencia entre representación estática y circuito interactivo, sin atribuirlos a la apariencia de los botones.

Si la página interactiva continúa sin responder, comprueba por este orden: ruta correcta, directiva de renderizado, registro de servicios en `Program.cs`, mapeo de endpoints, script en `App.razor` y errores de conexión que muestre el navegador. Un compilador con salida correcta no garantiza por sí solo la interacción de navegador: debes observar un clic que cambie el texto.

Reto 3.1 — Incrementos dobles

Enunciado: modifica una sola línea del método de evento de `Components/Pages/Interactiva.razor` para sumar dos unidades por pulsación, manteniendo intacta `Estatica.razor`. El objetivo es demostrar que el comportamiento depende de la lógica de C# y que la posibilidad de ejecutarla depende del modo de renderizado.

Código de solución para consulta:

```
private void Incrementar() => contador += 2;
```

Reemplaza temporalmente `contador++` por la línea anterior. Compila, navega a `/interactiva` y comprueba los valores 2 y 4 tras dos clics. **Control negativo:** `/estatica` sigue sin variar al pulsar. Restaura después `contador++`. La solución no añade nuevos endpoints, ni un formulario POST, ni persistencia. La modificación consiste únicamente en la operación realizada en el manejador.

Profundización opcional

Explica por qué el primer GET de una página interactiva sigue utilizando HTTP, aunque los eventos posteriores no se gestionen mediante una nueva acción MVC completa. Analiza qué puede ocurrir si el navegador pierde su conexión con el servidor mientras el contador está visible. No necesitas estudiar SignalR avanzado ni capturar los mensajes de protocolo: basta comprender que Interactive Server mantiene lógica y estado en el servidor y depende de esa comunicación.

Código completo que sostiene el laboratorio P3.1

En esta sección se incluyen los archivos completos de este estado, para que puedas estudiar **todo el código entregado y su función** antes de activar los cambios. Los archivos que no cambian respecto al punto anterior se mantienen y no se crean versiones paralelas.

CursoWeb.Blazor.csproj

Función del archivo. Define el SDK Microsoft.NET.Sdk.Web, el framework net8.0 y las opciones de análisis del compilador. No contiene lógica de catálogo, pero determina con qué herramientas puede compilar el alumno.

```
<Project Sdk="Microsoft.NET.Sdk.Web">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

Program.cs

Función del archivo. Crea el builder ASP.NET Core; registra componentes Razor y renderizado Interactive Server; construye el host, habilita archivos estáticos y antiforgery y mapea las rutas de componentes. La clase parcial final permite usar Program en pruebas.

```
using CursoWeb.Blazor.Components;

var builder = WebApplication.CreateBuilder(args);
// Servicios necesarios para componentes Razor y circuitos Interactive Server.
builder.Services.AddRazorComponents().AddInteractiveServerComponents();

var app = builder.Build();
if (!app.Environment.IsDevelopment()) app.UseExceptionHandler("/error");
app.UseStaticFiles();
app.UseAntiforgery();
// El modo Interactive Server necesita tanto servicios como endpoint.
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
app.Run();
public partial class Program { }
```

Components/App.razor

Función del archivo. Declara el documento HTML y sus recursos de hoja de estilos; incorpora HeadOutlet y Routes que dan entrada al árbol de componentes. La salida HTML depende de este documento raíz.

```
<!DOCTYPE html>
<html lang="es">
```

```
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <base href="/" />
    <link rel="stylesheet" href="app.css" />
    <HeadOutlet />
</head>
<body>
    <Routes />
    <script src="_framework/blazor.web.js"></script>
</body>
</html>
```

Components/Routes.razor

Función del archivo. Configura el router para el assembly de Program, selecciona la página mediante RouteView y aplica el layout. El fallback explica la respuesta al visitar una ruta inexistente.

```
<Router AppAssembly="typeof(Program).Assembly">
    <Found Context="routeData">
        <RouteView RouteData="routeData" DefaultLayout="typeof(MainLayout)" />
        <FocusOnNavigate RouteData="routeData" Selector="h1" />
    </Found>
    <NotFound>
        <LayoutView Layout="typeof(MainLayout)">
            <h1>404 | No existe la página solicitada</h1>
            <p><a href="/">Ir al inicio</a></p>
        </LayoutView>
    </NotFound>
</Router>
```

Components/_Imports.razor

Función del archivo. Agrupa los namespaces Razor, routing, render modes y tipos del proyecto disponibles para las páginas. Evita que cada componente tenga que declarar las mismas directivas using.

```
@using Microsoft.AspNetCore.Components
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Forms
@using static Microsoft.AspNetCore.Components.Web.RenderMode
@using CursoWeb.Blazor
```

```
@using CursoWeb.Blazor.Components
@using CursoWeb.Blazor.Components.Layout
```

Components/Layout/MainLayout.razor

Función del archivo. Define la cabecera, las rutas de navegación y el área @Body. El enlace Productos se mantiene comentado hasta P3.2 para que su activación constituya un paso nuevo observable.

```
@inherits LayoutComponentBase
<header><strong>Curso ASP.NET Core · Blazor B</strong>
<nav>
    <NavLink href="/" Match="NavLinkMatch.All">Inicio</NavLink>
    <NavLink href="/estatica">SSR estático</NavLink>
    <NavLink href="/interactiva">SSR interactivo</NavLink>
    @* PASO 3.2.2 (ACTIVAR): quita las marcas de comentario de este NavLink. *@

</nav></header>
<main>@Body</main>
```

Components/Pages/Home.razor

Función del archivo. Es el punto de entrada en la ruta raíz. Su contenido es informativo y facilita navegar a las dos páginas de comparación SSR.

```
@page "/"
<PageTitle>Curso Web | Blazor</PageTitle>
<h1>Aplicación B · Blazor Web App</h1>
<p>Partimos del catálogo MVC del módulo anterior, pero esta aplicación utiliza
componentes Razor.</p>
<p>En M03 los productos permanecen en memoria. No existe todavía una API ni una
base de datos.</p>
<p><a href="/estatica">Comparar SSR estático</a> · <a href="/interactiva">Comparar
interactividad</a></p>
```

Components/Pages/Estatica.razor

Función del archivo. Mantiene el botón con evento C# pero sin un render mode interactivo; el alumno observa que un clic de navegador no activa el método del circuito.

```
@page "/estatica"
<PageTitle>SSR estático</PageTitle>
<h1>P3.1 | Componente con SSR estático</h1>
<p>El servidor envía HTML. Este ejemplo no declara un modo de renderizado
```

```
interactivo.</p>
<p id="contador-estatico">Pulsaciones: @contador</p>
<button id="boton-estatico" @onclick="Incrementar">Pulsar (sin interacción)</
button>
<p class="mensaje">Aunque el marcado contenga @@onclick, sin renderizado
interactivo no se ejecuta el evento C# tras pulsar.</p>
@code {
    private int contador;
    private void Incrementar() => contador++;
}
```

Components/Pages/Interactiva.razor

Función del archivo. Incorpora InteractiveServer y contador local. Un clic invoca Incrementar y vuelve a representar el contador. El reto de incremento doble es una variante de una línea.

```
@page "/interactiva"
@rendermode InteractiveServer
<PageTitle>Interactive Server</PageTitle>
<h1>P3.1 | Componente interactivo en servidor</h1>
<p>El evento se procesa en el circuito de Blazor y la pantalla se actualiza sin
navegación HTTP.</p>
<p id="contador-interactivo">Pulsaciones: @contador</p>
<button id="boton-interactivo" @onclick="Incrementar">Pulsar (interactivo)</button>
@code {
    private int contador;
    private void Incrementar() => contador++;
    // RETO 3.1: sustituye la línea anterior por esta variante (no mantener ambas):
    // private void Incrementar() => contador += 2;
}
```

wwwroot/app.css

Función del archivo. Agrupa la presentación: tipografía, navegación, tarjetas y mensajes. Las clases usadas en el HTML Razor dependen de esta hoja; no son comportamiento C#.

```
:root{font-family:system-ui,-apple-system,"Segoe UI",sans-serif;color:#203047}
body{margin:0;background:#f7f9fc} header{background:#153656;color:white;padding:
1rem 1.5rem}
nav{display:flex;gap:1.4rem;flex-wrap:wrap}nav a{color:inherit}nav a.active{font-
weight:800;text-decoration:underline}
main{max-width:990px;margin:auto;padding:1.4rem;line-height:1.65} h1,h2{line-
height:1.25}
button{padding:.55rem 1.1rem;margin:.2rem;border:1px solid #527599;border-radius:.
```

```
4rem;cursor:pointer;background:white;color:#153656}
input{padding:.55rem;min-width:14rem}ul{padding-left:
1.25rem}.tarjetas{display:grid;grid-template-columns:repeat(auto-fit,minmax(230px,
1fr));gap:1rem}
.tarjeta{border:1px solid #c5d0dc;border-radius:.6rem;background:white;padding:
1rem} .mensaje{padding:.7rem;border-left:4px solid #3467a4;background:#edf4fb}
small{display:block;color:#506276}
```

Properties/launchSettings.json

Función del archivo. Define perfiles y URLs de lanzamiento para herramientas de desarrollo. No reemplaza al mapeo de endpoints; facilita arrancar la aplicación en el entorno local.

```
{
  "profiles": {
    "CursoWeb.Blazor": {
      "commandName": "Project",
      "launchBrowser": true,
      "applicationUrl": "http://localhost:5092",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

Estos once archivos establecen el proyecto inicial completo. El trabajo del alumno consiste en leerlos, ejecutarlos y vincular su responsabilidad con los resultados de los clics y la navegación. No se espera que memorice la plantilla: debe comprender por qué el host, el router, el layout, las páginas y los estilos son partes diferentes del mismo programa.

Cuaderno de profundización P3.1 — Distinguir cinco tipos de evidencia

El itinerario esencial ya ha demostrado que el modo de renderizado determina si la interacción C# está disponible. La siguiente actividad permite profundizar en **cómo sabemos que algo funciona**. No requiere archivos adicionales ni obliga a modificar de forma permanente el proyecto. Realízala después de completar 3.1.5 y antes de pasar a P3.2, o consérvala como ejercicio de consulta.

Primer tipo de evidencia: el host está iniciado

Abre una consola en la carpeta del proyecto 3.1 y ejecuta `dotnet run --no-launch-profile --urls http://localhost:5092`. Observa el mensaje del host que indica que Kestrel está escuchando. Esa salida demuestra que el proceso se ha iniciado, pero **no** demuestra que la página tenga un evento interactivo correcto. Es posible que un servidor responda HTTP y que después un componente falle al renderizar; por eso la siguiente observación debe realizarse desde el navegador.

Visita `/` y comprueba que aparece el texto de la aplicación B. Si recibes «No se puede conectar», revisa la URL y el puerto, y confirma que no estás ejecutando otro proyecto. Si aparece una pantalla con el catálogo MVC de M02, detén el proceso anterior y vuelve a iniciar 3.1. Esta incidencia se resuelve identificando el **host correcto**, no cambiando el código de un contador.

Segundo tipo: una URL corresponde a un componente

Abre `/estatica` y `/interactiva` en pestañas distintas. En cada una observa su encabezado y el código fuente de la página. La primera corresponde a `Estatica.razor` y la segunda a `Interactiva.razor`. Ambas declaran una ruta mediante `@page`. La respuesta del servidor confirma que el router puede seleccionar los componentes; no demuestra todavía que el botón produzca un evento.

Como caso negativo, abre `/no-existe`. El router presenta la rama de página no encontrada. Contrasta este resultado con una aplicación que no arranca: en uno puedes recibir una respuesta HTML de la propia aplicación y en el otro no llegas a establecer comunicación con el host. Escribe dos frases que permitan a otro alumno distinguir los síntomas sin acceder al código.

Tercero: el HTML contiene un botón y un estado inicial

En cada página identifica el párrafo que muestra las pulsaciones. El HTML inicial presenta un valor cero. Eso demuestra que Razor ha podido evaluar el campo `contador` durante la representación. No demuestra que el método de evento pueda ejecutarse **después** de cargar la página. Este detalle es importante porque un test automatizado que solo descargara la dirección inicial no verificaría la interactividad de Blazor.

Abre las herramientas de desarrollo, inspecciona los elementos `#boton-estatico` y `#boton-interactivo` y observa su parecido. Comparten la forma de describir un evento en Razor, pero participan en modos de ejecución diferentes. Un estilo visual parecido tampoco equivale a un comportamiento idéntico; por eso necesitamos el cuarto tipo de evidencia.

Cuarto: el navegador ejecuta una interacción real

Pulsa el botón estático dos veces y observa que el número permanece en cero. Después pulsa el interactivo dos veces y comprueba que aparece dos. Esta es la evidencia que demuestra el objetivo principal. Anota además si la URL de la página cambió durante la operación; el contador se actualiza sin que debamos crear una ruta específica para cada valor.

Si el botón interactivo no responde, no declares éxito basándote en la existencia de `@onclick` en el archivo. Comprueba `@rendermode`, los servicios y el script, y examina la consola del navegador. La prueba positiva exige un cambio visible generado como consecuencia de un clic concreto.

Quinto: continuidad del estado y dependencia de la sesión

Con el contador interactivo en tres, recarga completamente la página. Observa si comienza una nueva instancia que muestra de nuevo el valor inicial. En nuestro laboratorio no hay base de datos ni servicio de persistencia que conserve ese valor. Explica por qué una recarga y un clic son operaciones distintas: una puede crear una nueva instancia de componente, mientras el otro modifica el estado de la instancia activa.

No es necesario medir la duración exacta del circuito ni estudiar procedimientos de reconexión avanzados. El objetivo es distinguir **estado de componente** de **dato persistente** y reconocer que Interactive Server necesita comunicación con el servidor para los eventos.

Resolución de tres incidencias simuladas

Incidencia A. El alumno observa el contador estático en cero y asegura que el método C# es incorrecto. La explicación razonada es que el método puede

estar bien escrito, pero la página no declara interactividad; el problema es de **modo de ejecución**, no de aritmética.

Incidencia B. El alumno ve `/interactiva`, pulsa y no ocurre nada tras perder conexión. Debe verificar el circuito y los recursos del navegador antes de cambiar el código de `Incrementar`. No puede garantizar que el evento haya llegado al servidor.

Incidencia C. El alumno pulsa, el número llega a cinco, recarga y vuelve a cero. No se trata de un error de `@bind` ni de un problema de SQL; la variable es un campo local y no tiene almacenamiento permanente.

Elabora una pequeña tabla personal con la causa y la evidencia de cada incidencia. Compara tus respuestas con la teoría 3.1 sobre SSR, Interactive Server y vida del estado. Si sabes explicar por qué una página puede compilar, responder HTTP y aun así no atender un clic C#, has alcanzado una comprensión que se trasladará a todos los puntos siguientes.

Comparación transversal con M02

En el CRUD MVC, una respuesta HTTP correcta después de un POST podía terminar en una redirección a Detalle y una consulta a LocalDB. En el contador Blazor, la acción no necesita navegar ni guardar. No confundas «respuesta del servidor» con «persistencia»: son responsabilidades diferentes. M03 construirá una interfaz interactiva y M04 añadirá transporte remoto; cada fase tendrá pruebas adecuadas a lo que verdaderamente ejecuta.

P3.2. Página Productos y navegación con NavLink

Objetivo y estado inicial

El proyecto `M03_BLAZOR/PROYECTO/3.2/CursoWeb.Blazor` conserva las páginas anteriores y añade el modelo `Producto` y la página `/productos`. La ruta **ya funciona**, pero el enlace del menú permanece comentado para aprender que declarar un endpoint y ofrecer navegación son responsabilidades diferentes.

3.2.1. Abrir el proyecto y probar la ruta directamente

Detén el host de P3.1 y abre `M03_BLAZOR/PROYECTO/3.2/CursoWeb.Blazor/CursoWeb.Blazor.csproj`. Compila y ejecuta en el puerto 5092. Sin utilizar el menú, abre directamente `http://localhost:5092/productos`. Debes encontrar un listado de Cuaderno, Lápiz y Carpeta. Si la dirección funciona pero todavía no existe el enlace «Productos» en la cabecera, ese es exactamente el estado inicial esperado.

Abre `Components/Pages/Productos.razor` y encuentra las directivas `@page "/productos"` y `@rendermode InteractiveServer`. La primera declara la dirección. La segunda anticipa la interactividad que añadiremos después, aunque por ahora solo representemos una lista.

3.2.2. Activar el enlace preparado en el layout

Abre `Components/Layout/MainLayout.razor` y localiza la marca `PASO 3.2.2 (ACTIVAR)`. Justo debajo aparece:

```
@* <NavLink href="/productos">Productos</NavLink> *@
```

Elimina los delimitadores de comentario, conservando:

```
<NavLink href="/productos">Productos</NavLink>
```

Guarda y recompila. Abre `http://localhost:5092/`. El menú debe mostrar «Productos». Haz clic, observa la ruta final y regresa al inicio. El elemento

`NavLink` ofrece el enlace y la indicación de navegación activa, pero **no crea por sí mismo la página**; esta ya existía gracias a la directiva `@page`.

3.2.3. Leer el código de modelo y lista

Abre `Models/Producto.cs` y comprueba la declaración del record con Id, Nombre y Precio. En `Components/Pages/Productos.razor` localiza:

```
<ul id="lista-productos">
    @foreach (var producto in productos)
    {
        <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
    }
</ul>
```

Dentro de `@code` se crea una colección con tres elementos. Relaciona la lista C# con los tres elementos `li` del HTML representado. Abre las herramientas del navegador e inspecciona `id="lista-productos"`. No existe ninguna petición SQL ni HTTP a una API para rellenarla: el estado pertenece al componente.

Ejercicio de diagnóstico: si el menú no muestra Productos, investiga el layout; si aparece el menú pero `/productos` no existe, investiga `@page` y el router. No cambies los archivos de M02 para arreglar un problema de esta nueva aplicación.

3.2.4. Comparar tres nombres parecidos que representan ideas distintas

Explica la diferencia entre `Views/Productos/Index.cshtml` de MVC, `Pages/Comparacion.cshtml` de Razor Pages y `Components/Pages/Productos.razor` de Blazor. Los tres generan contenido de interfaz a partir de Razor, pero el primero se elige desde una acción MVC, el segundo usa handlers de PageModel y el tercero es un componente que puede tener renderizado interactivo. Esta comparación enlaza expresamente con los módulos anteriores y evita una confusión que suele aparecer al comenzar con Blazor.

Ejercicio autónomo 3.2 — Añadir una fila controlada

Enunciado: añade un producto «Regla», con Id 4 y precio 2.50, dentro de la colección que declara `Productos.razor`. No dupliques el elemento HTML ni crees otro archivo. **Código que debes escribir:** incorpora un cuarto elemento:

```
new(4, "Regla", 2.50m)
```

Compila y recarga `/productos`. Deben aparecer **cuatro elementos de lista**. El formato del precio debe seguir la expresión `ToString("0.00")`. Si solo ves tres elementos, verifica que editaste el proyecto 3.2 que estás ejecutando. Si el precio se representa con un formato diferente, investiga cómo se convierte el decimal a texto. Después recupera la colección inicial de tres elementos para continuar con P3.3 tal como se entrega.

Profundización opcional

Cambia temporalmente el texto visible del enlace de menú sin modificar su `href`. Observa que la ruta sigue siendo la misma. Ahora imagina que cambiaras `@page "/productos"` a otro nombre sin actualizar el menú: ¿qué enlace quedaría roto? No hace falta alterar definitivamente la ruta; basta explicar que la dirección es un contrato distinto del texto que la representa. Este criterio es útil en MVC, Razor Pages y Blazor.

Código completo de la ampliación P3.2

En esta sección se incluyen los archivos completos de este estado, para que puedas estudiar **todo el código entregado y su función** antes de activar los cambios. Los archivos que no cambian respecto al punto anterior se mantienen y no se crean versiones paralelas.

Models/Producto.cs

Función del archivo. Declara el record del dominio de laboratorio: Id, Nombre y Precio. Toda la evolución posterior comparte este mismo tipo.

```
namespace CursoWeb.Blazor.Models;
/// <summary>Producto de laboratorio: sin persistencia ni DTO remoto en M03.</
summary>
public sealed record Producto(int Id, string Nombre, decimal Precio);
```

Components/Pages/Productos.razor

Función del archivo. Declara /productos, la lista de tres records y el foreach de Razor; cada li refleja un producto y su precio formateado en euros.

```
@page "/productos"
@rendermode InteractiveServer
<PageTitle>Productos</PageTitle>
<h1>P3.2 | Página Productos</h1>
<p>Catálogo de ejemplo en memoria. Todavía no consumimos una API.</p>
<ul id="lista-productos">
    @foreach (var producto in productos)
    {
        <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
    }
</ul>
@code {
    private readonly List<Producto> productos =
    [
        new(1, "Cuaderno", 4.50m),
        new(2, "Lápiz", 1.20m),
        new(3, "Carpeta", 3.75m)
        // APLICACIÓN 3.2: añade una coma tras Carpeta y activa esta cuarta fila.
        // , new(4, "Regla", 2.50m)
    ];
}
```

Components/Layout/MainLayout.razor

Función del archivo. Contiene el NavLink de Productos preparado y comentado. El paso P3.2 lo activa sin reconstruir ni sustituir la navegación de P3.1.

```
@inherits LayoutComponentBase
<header><strong>Curso ASP.NET Core · Blazor B</strong>
<nav>
    <NavLink href="/" Match="NavLinkMatch.All">Inicio</NavLink>
    <NavLink href="/estatica">SSR estático</NavLink>
    <NavLink href="/interactiva">SSR interactivo</NavLink>
    @* PASO 3.2.2 (ACTIVAR): quita las marcas de comentario de este NavLink. *@
    @* <NavLink href="/productos">Productos</NavLink> *@
</nav></header>
<main>@Body</main>
```

El estudiante debe conservar intactos los archivos del host y las dos páginas de P3.1. El ejercicio modifica MainLayout para activar la navegación y introduce un modelo y una página; no reimplementa Program ni deshace el router.

Cuaderno de profundización P3.2 — Investigar navegación, modelos y representación

Este recorrido opcional se realiza sobre `PROYECTO/3.2/CursoWeb.Blazor` después de activar NavLink. Su objetivo es reconocer qué archivo cambia cada comportamiento de la pantalla. No añade otra página obligatoria ni necesita crear servicios.

Escenario A: ruta correcta, enlace ausente

Abre directamente `/productos`. Si la página funciona, tienes evidencia de que el endpoint existe. Después ve a `/` y comprueba la navegación. Si el enlace no está, revisa `Components/Layout/MainLayout.razor` y localiza la línea marcada `PASO 3.2.2`. Asegúrate de haber retirado los delimitadores del comentario. No renombres `Productos.razor` ni modifiques su directiva de ruta para solucionar un menú que está comentado.

Este caso muestra una relación de **oferta de navegación**: el usuario puede conocer una URL aunque no aparezca en el menú. Lo mismo ocurría en MVC con las acciones que no tenían enlace visible. La decisión de publicar una ruta y la decisión de mostrarla como opción de interfaz deben estar coordinadas, pero no son la misma operación.

Escenario B: el menú apunta a una página que no existe

Cambia **temporalmente** la dirección del enlace por `/catalogo-inventado`, manteniendo `@page "/productos"` en la página. Compila y haz clic en el menú. Debes observar el comportamiento de ruta no encontrada. El resultado no indica que los tres productos hayan desaparecido de la colección; indica que el navegador fue dirigido a una dirección que el router no puede resolver en este proyecto.

Restablece `href="/productos"` y comprueba que el listado vuelve a estar disponible. La solución razonada es mantener coherencia entre la URL pública y los enlaces; no es crear una segunda lista de productos dentro del layout.

Escenario C: los datos cambian, pero el marco sigue estable

En `Components/Pages/Productos.razor` modifica temporalmente el precio de Cuaderno y recompila. La pantalla mostrará el nuevo valor mientras el menú y la estructura visual permanecen. Esa separación demuestra que `MainLayout.razor` no necesita conocer el precio del producto. Si tu cambio modifica el menú por accidente, has editado el archivo equivocado o mezclado responsabilidades.

Repite con un cambio puramente visual: modifica el texto fijo del encabezado de la página. No necesitas tocar el record Producto para ello. Vuelve después a los textos y precios iniciales.

Escenario D: un cambio de nombre rompe una referencia de C#

Si cambiaras la propiedad `Nombre` del record por otra sin adaptar las expresiones Razor que la consultan, tendrías un problema de compilación o referencias, aunque la ruta continuara siendo correcta. No es necesario ejecutar esa modificación destructiva; basta localizar dónde se utiliza `producto.Nombre` y explicar la dependencia con `Models/Producto.cs`.

La diferencia con los fallos anteriores importa: en A fallaba la navegación visible; en B, el destino de la ruta; en C, solo cambiaba el dato o el texto representado; y en D cambiaría el contrato de C# que el componente utiliza.

Diagnóstico paso a paso del HTML

Inspecciona el elemento `ul` de `/productos`. Debe contener tres `li`. Los elementos son el resultado del bucle Razor, no una respuesta JSON de API. Si la lista aparece vacía, comprueba que los tres objetos siguen definidos en `@code`. Si el HTML contiene el mismo producto varias veces, investiga las entradas de la colección y el cuerpo del bucle.

Observa que `ToString("0.00")` se aplica al representar el precio. La representación cultural puede afectar a separadores decimales según el contexto. El alumno no necesita rediseñar la internacionalización completa del curso, pero debe saber que los datos numéricos y su texto visible son entidades distintas. En M02 ya se examinó la conversión de entrada; aquí hablamos principalmente de formato de salida.

Interpretación del modo interactivo sin necesidad inmediata de eventos

`Productos.razor` declara `@rendermode InteractiveServer` aunque el estado inicial solo represente una lista. Eso no significa que la página ya tenga un botón con lógica propia. El modo prepara la capacidad interactiva; los eventos específicos se añadirán en P3.4. La diferencia se parece a tener una acción MVC disponible sin que todos los enlaces la invoquen: disponer de infraestructura no determina por sí solo las operaciones concretas que la aplicación realiza.

Solución razonada del ejercicio de transferencia

Si deseas agregar una ruta nueva, crea otro componente de página que declare `@page` y actualiza la navegación cuando tenga sentido. Si quieres cambiar el menú, edita el layout. Si deseas cambiar un producto de esta muestra fija, edita la colección de la página. Si necesitas cambiar el estilo general, edita `wwwroot/app.css`. Y si quieres modificar cómo se representa **cada producto** sin repetir HTML, espera al siguiente punto y utiliza TarjetaProducto.

La finalidad no es aprender una lista de carpetas de memoria. Es descubrir que cada carpeta y archivo responde a una decisión técnica diferente, y que diagnosticar bien ahorra cambios innecesarios.

P3.3. Componente TarjetaProducto, parámetros y composición

Objetivo y preparación

Ahora convertimos una lista HTML sencilla en tres tarjetas generadas mediante un componente reutilizable. El estado inicial de `M03_BLAZOR/PROYECTO/3.3/CursoWeb.Blazor` sigue compilando y mostrando los productos mediante `<ul>`. El proyecto incluye además `Components/TarjetaProducto.razor` y una variante comentada de composición. Necesitamos activar la segunda **sustituyendo** el bloque antiguo para no duplicar la interfaz.

3.3.1. Verificar el estado de salida de P3.2

Detén P3.2, abre el proyecto 3.3, compila y ejecuta. Abre `/productos`. Debes encontrar una lista de tres elementos. El objetivo inmediato no es añadir datos nuevos; es cambiar **cómo** se representan los mismos tres objetos.

En `Components/Pages/Productos.razor` localiza el comentario `PASO 3.3.2` y el bloque activo que comienza con `<ul id="lista-productos">`. Más abajo encuentra `PASO 3.3.3 (ACTIVAR)`, que incluye una variante de tarjetas dentro de un comentario Razor.

3.3.2. Comprender el contrato de la tarjeta

Abre `Components/TarjetaProducto.razor`. Encontrarás un artículo con nombre, precio e identificador:

```
<article class="tarjeta" data-producto-id="@Producto.Id">
    <h2>@Producto.Nombre</h2>
    <p>Precio: @Producto.Precio.ToString("0.00") €</p>
    <small>Referencia @Producto.Id</small>
</article>
```

Debajo aparece:

```
[Parameter, EditorRequired]
public Producto Producto { get; set; } = default!;
```

Explica por qué `[Parameter]` establece un contrato de entrada. La tarjeta no conoce la colección completa, no tiene que consultar una API y no declara

`@page`. Solo necesita el producto que recibe del componente padre. La marca `EditorRequired` ayuda a las herramientas a identificar que se espera ese parámetro; no introduce por sí sola un mecanismo de validación HTTP.

3.3.3. Sustituir la lista por tres tarjetas

En `Components/Pages/Productos.razor`, comenta temporalmente el bloque de lista completo, desde `<ul id="lista-productos">` hasta `</ul>`, incluidos los elementos de iteración que contiene. Activa el bloque siguiente retirando `@*` y `*@`:

```
<div class="tarjetas" id="tarjetas-productos">
    @foreach (var producto in productos)
    {
        <TarjetaProducto Producto="producto" />
    }
</div>
```

Compila y abre `/productos`. **Resultado positivo:** tres tarjetas, una por producto, con sus nombres, precios e identificadores. En el inspector del navegador busca `data-producto-id="1"`, `data-producto-id="2"` y `data-producto-id="3"`. El atributo ayuda a comprobar qué producto recibió cada tarjeta.

Caso negativo de transformación: si conservaste activo el bloque `<ul>` además del nuevo bloque, aparecerán dos representaciones del catálogo. Eso no demuestra que `[Parameter]` haya fallado; indica que has dejado dos bucles activos. Deja solo la versión de tarjetas.

3.3.4. Explicar el vínculo entre padre e hijo

En el padre, la variable `producto` procede del `foreach`. En el hijo, la propiedad `Producto` recibe ese objeto. La expresión `Producto="producto"` es el punto donde las dos piezas se encuentran. Si sustituyes temporalmente la variable por `productos[0]`, las tres tarjetas pueden representar el mismo objeto Cuaderno aunque haya tres iteraciones. Restablece `Producto="producto"` y comprueba que vuelven los tres productos distintos. La prueba demuestra que el componente reutilizable necesita datos correctos del padre; no crea por sí mismo una lista independiente.

3.3.5. Observar un cambio centralizado

En `Components/TarjetaProducto.razor` sustituye temporalmente el texto «Precio:» por «Importe:» y compila. El cambio aparece en las tres tarjetas. No es necesario modificar el padre varias veces. Después restaura el texto original. Esta observación justifica la extracción del componente: una misma responsabilidad de representación está concentrada en un lugar.

Si los elementos se dibujan sin tarjetas visuales, comprueba la clase `tarjeta` y la carga de `wwwroot/app.css`. Si no se muestran los nombres correctos, inspecciona los valores que el padre transmite. No confundas un problema de estilos con un problema de parámetros.

Reto 3.3 — Indicar productos que superan un precio

Enunciado: modifica exclusivamente `Components/TarjetaProducto.razor` para que un producto cuyo precio sea superior a cuatro euros muestre el texto «Precio superior a 4 euros». Se trata de un ejercicio autónomo de representación condicional, no de una nueva regla de negocio o de una llamada a un servicio.

Código de solución razonada:

```
@if (Producto.Precio > 4m)
{
    <p class="mensaje">Precio superior a 4 euros</p>
}
```

Escribe el bloque dentro del artículo, después del precio. Compila y observa el resultado: Cuaderno tiene precio 4.50 y debe mostrar la advertencia; Lápiz y Carpeta no. **Caso negativo:** no deben aparecer mensajes en productos que cuestan menos de cuatro euros. Para explorar la frontera entre `>` y `>=`, puedes cambiar temporalmente el precio de Carpeta a exactamente 4.00, observar el comportamiento y restaurarlo.

Explicación: esta responsabilidad pertenece al hijo porque es una variación de cómo se representa **un** producto. No hace falta duplicar el bucle en el padre ni crear una vista Razor MVC. El componente recibe el objeto y decide qué HTML adicional producir.

Profundización opcional: reutilización y responsabilidades

Imagina que la página Productos se utiliza más adelante para mostrar resultados de una API. ¿Debería `TarjetaProducto` hacer por sí misma una petición HTTP cada vez que se representa? En nuestro diseño, no: el padre o un servicio coordinador obtiene datos, y las tarjetas reciben los objetos que deben mostrar. Este planteamiento permite reutilizar la representación sin acoplarla al origen de datos y prepara el servicio de P3.6.

Relación con la teoría: 3.3 desarrolla el contrato `[Parameter]`, la composición y la propiedad de los datos. Al terminar debes poder explicar por qué la tarjeta no necesita una ruta `@page` para funcionar dentro de la página.

Código completo que se transforma en P3.3

En esta sección se incluyen los archivos completos de este estado, para que puedas estudiar **todo el código entregado y su función** antes de activar los cambios. Los archivos que no cambian respecto al punto anterior se mantienen y no se crean versiones paralelas.

Components/TarjetaProducto.razor

Función del archivo. Extrae el article de la tarjeta con su Id en data-producto-id, el título y el precio, y define un parámetro obligatorio de entrada.

```
<article class="tarjeta" data-producto-id="@Producto.Id">
    <h2>@Producto.Nombre</h2>
    <p>Precio: @Producto.Precio.ToString("0.00") €</p>
    <small>Referencia @Producto.Id</small>
    @* RETO 3.3: eliminar los delimitadores del bloque siguiente. *@
    @*
    @if (Producto.Precio > 4m)
    {
        <p class="mensaje">Precio superior a 4 euros</p>
    }
    *@
</article>
@code {
    [Parameter, EditorRequired] public Producto Producto { get; set; } = default!;
}
```

Components/Pages/Productos.razor

Función del archivo. Conserva la colección y la ruta existentes, pero permite sustituir el ul de P3.2 por el foreach que instancia TarjetaProducto. La nueva función es la composición; no se crea otra fuente de datos.

```
@page "/productos"
@rendermode InteractiveServer
<PageTitle>Productos</PageTitle>
<h1>P3.3 | Composición y parámetros</h1>
<p>Tres productos en memoria. Cada tarjeta es una instancia del mismo componente.</
p>
@* PASO 3.3.2: el siguiente listado ya funciona en el estado inicial. *@
<ul id="lista-productos">
    @foreach (var producto in productos)
    {
        <li>@producto.Nombre — @producto.Precio.ToString("0.00") €</li>
    }
</ul>
@* PASO 3.3.3 (ACTIVAR): comenta el bloque <ul> anterior y quita los
   delimitadores del bloque siguiente, que pasa cada objeto mediante [Parameter].
*@
@*
<div class="tarjetas" id="tarjetas-productos">
    @foreach (var producto in productos)
    {
        <TarjetaProducto Producto="producto" />
    }
</div>
*@
@code {
    private readonly List<Producto> productos =
    [
        new(1, "Cuaderno", 4.50m),
        new(2, "Lápiz", 1.20m),
        new(3, "Carpeta", 3.75m)
    ];
}
```

La transformación de P3.3 reemplaza un listado HTML por instancias del mismo TarjetaProducto, manteniendo exactamente la colección y la ruta de P3.2. Cada producto aporta su objeto al parámetro tipado, y el componente hijo es responsable del marcado de la tarjeta.

Cuaderno de profundización P3.3 — Contrato del componente y pruebas de identidad

En el recorrido principal has sustituido el listado por tres instancias de TarjetaProducto. Esta ampliación opcional permite comprobar qué tan sólida es esa composición sin añadir API ni base de datos. Utiliza `PROYECTO/3.3/CursoWeb.Blazor` con el bloque de tarjetas activo y el `ul` original desactivado.

Caso A: tres tarjetas visibles, pero un único dato

Reproduce temporalmente la expresión equivocada `Producto="productos[0]"` en el padre. Verás tres instancias visuales que muestran la información de Cuaderno. Es importante entender que se han creado **tres componentes**, pero se les ha transmitido **el mismo objeto**. Por eso contar tarjetas no basta como prueba de que la aplicación representa una colección correctamente.

Restablece `Producto="producto"` y comprueba que los atributos `data-producto-id` son diferentes. Un diagnóstico riguroso observa tanto la cantidad de elementos como la identidad y el contenido representado por cada uno.

Caso B: una tarjeta recibe un contrato incompleto

El hijo declara `[Parameter, EditorRequired] public Producto Producto`. Si el padre intentara representar `<TarjetaProducto />` sin proporcionar el parámetro, estaríamos rompiendo un contrato esperado del componente. No necesitas dejar esa versión en el proyecto; basta explicar que la tarjeta utiliza `Producto.Id` y `Producto.Nombre`, por lo que no puede producir una representación correcta sin recibir un producto.

`EditorRequired` ayuda a señalar el requisito desde las herramientas. No debes describirlo como una garantía absoluta de ejecución segura: el código sigue siendo responsable de gestionar entradas y estados válidos. La anotación comunica intención al autor del componente y al compilador o editor.

Caso C: el marcado cambia sin alterar la lógica del padre

En TarjetaProducto cambia temporalmente el texto «Referencia» por «Código». Compila y observa el resultado en las tres tarjetas. El padre no necesita saber cómo se etiqueta el identificador dentro de cada tarjeta. Después restaura el

texto. Esa pequeña modificación muestra de forma concreta el beneficio de separar responsabilidades, incluso cuando todavía no hay métodos complejos ni servicios.

Ahora cambia temporalmente el valor del precio de Lápiz **en el padre**. Su tarjeta debe reflejarlo sin que cambies el código del hijo. El contrato se conserva: TarjetaProducto recibe un objeto y lo representa. No sería adecuado copiar el precio a una constante privada dentro de la tarjeta, porque dejaría de reflejar el dato que recibe.

Caso D: el estilo falta, pero los datos son correctos

Abre el inspector del navegador. Si todos los elementos `article` existen con los Id correctos, pero carecen del aspecto de tarjeta, investiga `wwwroot/app.css` y la clase `tarjeta`. Esa incidencia se refiere a presentación visual, no al flujo de parámetros. Cambiar `Producto` por otro tipo no cargaría una hoja de estilos perdida.

También es posible lo contrario: las tarjetas pueden tener bordes y distribución correcta, pero mostrar datos equivocados. Entonces los estilos funcionan y la causa puede encontrarse en la expresión del padre o en el contenido de su lista.

Caso E: reorganizar elementos sin reescribir componentes

Imagina que el catálogo pasa a tener cuatro elementos. El `foreach` representará cuatro instancias del mismo componente. No es necesario copiar cuatro veces el bloque `<article>`. Esta es una ventaja funcional de la composición, además de su valor de mantenimiento. Si más adelante se reorganiza la lista, Blazor tiene mecanismos como `@key` para conservar identidad de componentes en escenarios donde importa; no añadimos esa dificultad al itinerario esencial, que trabaja con una lista pequeña y controlada.

Ejercicio complementario de marcado condicionado

El reto principal propone mostrar una marca cuando el precio supera cuatro euros. Como profundización, prueba tres valores para esa frontera: 3.99, 4.00 y 4.01. Con la condición `Precio > 4m`, solo 4.01 debe mostrar la marca. Con `Precio >=`

`4m`, también 4.00 la mostraría. Restaura los datos originales después de comprobarlo.

La diferencia importa porque el alumno no debe aceptar una prueba únicamente por mostrar el mensaje en Cuaderno. Es posible que una condición incorrecta produzca el mismo resultado para los tres datos iniciales. Diseñar valores alrededor de una frontera es una técnica de pruebas que ya conoces de C# y aplicas ahora al comportamiento de un componente.

Interpretación final

El padre decide **qué elementos** forman parte de la colección y el orden en que aparecen; el hijo decide **cómo se representa** cada elemento recibido. Esa separación no impide que la tarjeta tenga posteriormente un botón: solo significa que, cuando deba informar de una acción, necesitaremos un contrato de evento para no modificar silenciosamente el estado del padre. Ese será el problema central de P3.5.

P3.4. Eventos, @onclick, @bind y estado de componentes

Laboratorio principal acumulativo P3.4 — Ampliar P3.3 sin perder sus componentes

Situación didáctica. En P3.3 hemos construido un catálogo que representa objetos Producto mediante un componente TarjetaProducto con su parámetro tipado. En P3.4 ese componente se **conserva**; no volvemos a colocar el artículo HTML de cada producto directamente dentro de la página. Vamos a añadir primero selección, después entrada enlazada y creación, finalmente edición y consulta. Este es el mismo proyecto B y la ruta sigue siendo /productos. La novedad de este capítulo es la gestión del estado interactivo.

3.4.1 — Abrir el estado heredado y reconocer cada archivo

Abre la carpeta M03_BLAZOR/PROYECTO/3.4/CursoWeb.Blazor en Visual Studio 2026. Comprueba que continúan las páginas /estatica e /interactiva de P3.1, el enlace de navegación de P3.2 y el modelo y la tarjeta de P3.3. El alumno debe comprender que la instantánea 3.4 **parte del resultado de 3.3**, no de un segundo catálogo.

El modelo de datos, Models/Producto.cs, es este archivo completo:

```
namespace CursoWeb.Blazor.Models;
/// <summary>Producto de laboratorio: sin persistencia ni DTO remoto en M03.</
summary>
public sealed record Producto(int Id, string Nombre, decimal Precio);
```

Id permite seleccionar, editar y conservar identidad. Nombre aparece tanto en el listado como en el editor; Precio se representa en cada tarjeta pero no será modificado por el editor de nombre. El record es apropiado para ensayar una sustitución mediante with.

El componente reutilizado de 3.3 continúa íntegro y no se reescribe en 3.4:

```
<article class="tarjeta" data-producto-id="@Producto.Id">
    <h2>@Producto.Nombre</h2>
    <p>Precio: @Producto.Precio.ToString("0.00") €</p>
```

```
    <small>Referencia @Producto.Id</small>
</article>
@code {
    [Parameter, EditorRequired] public Producto Producto { get; set; } = default!;
}
```

Fíjate en el atributo data-producto-id: representa la identidad del producto, no su índice en la lista. El parámetro Producto recibe el objeto de la iteración del padre. Toda la representación HTML del artículo vive en este archivo. El padre puede envolver la tarjeta en un contenedor para añadir controles, pero **no duplica el artículo**.

3.4.2 — Consultar el archivo completo de la página antes de activar nada

El archivo Components/Pages/Productos.razor se entrega con el estado inicial compilable. El siguiente bloque contiene el **100 % del archivo entregado para este punto**, incluidos los marcadores de código comentado. Léelo siguiendo este orden: ruta y modo interactivo; títulos y selección; tarjeta y acciones; controles de edición y alta; colección de ejemplo; métodos comentados para la actividad.

```
@page "/productos"
@rendermode InteractiveServer
<PageTitle>Eventos</PageTitle>
<h1>P3.4 | Estado y eventos</h1>
<p>Los datos existen únicamente durante la sesión interactiva de este componente.</
p>
<p id="seleccion">Seleccionado: @seleccion</p>
@* RETO INTEGRADOR 3.4.6: activa los tres controles y su lógica C#;
   el filtro se aplica inmediatamente al escribir y no modifica la colección. *@
@* <label>Buscar por nombre: <input id="buscar" @bind="busqueda"
@bind:event="oninput" /></label> *@
@* <label>Precio máximo: <input id="precio-maximo" type="number" min="0" step="0.5"
@bind="precioMaximo" /></label> *@
@* <button id="alternar-orden" @onclick="AlternarOrden">Invertir orden por precio</
button> *@
@* <p id="resumen-filtrado">Mostrando @ProductosVisibles.Count() de
@productos.Count productos</p> *@
@* RETO INTEGRADOR 3.4.6: sustituir solamente la expresión foreach
   original por la variante siguiente, conservando todo el artículo y botones. *@
@* @foreach (var producto in ProductosVisibles) *@
<div class="tarjetas" id="tarjetas-productos">
    @foreach (var producto in productos)
    {
```

```
        <div class="tarjeta-con-acciones" data-elemento-id="@producto.Id">
            @* Conservamos TarjetaProducto de P3.3: el padre administra eventos en
P3.4. *@
            <TarjetaProducto Producto="producto" />
            @* PASO 3.4.2 (ACTIVAR): descomenta este botón al activar Seleccionar.
*@
            @* <button @onclick="() => Seleccionar(producto.Id)">Seleccionar</
button> *@
            @* PASO 3.4.5: sustituir el botón anterior, no añadir un segundo. *@
            @* <button @onclick="() =>
SeleccionarParaEditar(producto.Id)">Seleccionar</button> *@
        </div>
    }
</div>
@* PASO 3.4.3 (ACTIVAR): descomenta el input tras activar el campo nuevoNombre. *@
@* <label>Nombre nuevo: <input id="nombre-nuevo" @bind="nuevoNombre" /></label> *@
@* PASO 3.4.4 (ACTIVAR): descomenta el botón tras activar Agregar. *@
@* <button id="agregar" @onclick="Agregar">Añadir producto</button> *@
@* PASO 3.4.5: activa juntos el campo y el botón tras activar el método de edición.
*@
@* <label>Editar nombre: <input id="editar-nombre" @bind="nombreEditado" /></label>
*@
@* <button id="guardar-cambio" @onclick="GuardarCambio">Guardar nombre</button> *@
@* RETO 3.4 (ACTIVAR): botón para deshacer la selección. *@
@* <button id="limpiar" @onclick="Limpiar">Limpiar selección</button> *@
@code {
    private readonly List<Producto> productos =
    [
        new(1, "Cuaderno", 4.50m),
        new(2, "Lápiz", 1.20m),
        new(3, "Carpeta", 3.75m)
    ];
    private string seleccion = "ninguno";

    /* PASO 3.4.2 INICIO
    private void Seleccionar(int id)
    {
        seleccion = productos.First(p => p.Id == id).Nombre;
    }
    PASO 3.4.2 FIN */

    /* PASO 3.4.3 INICIO
    private string nuevoNombre = string.Empty;
    PASO 3.4.3 FIN */

    /* PASO 3.4.4 INICIO
    private int siguienteId = 4;
    private void Agregar()
    {
        if (string.IsNullOrWhiteSpace(nuevoNombre))
```

```
            return;
        productos.Add(new Producto(siguienteId++, nuevoNombre.Trim(), 5m));
        nuevoNombre = string.Empty;
    }
    PASO 3.4.4 FIN */


    /* PASO 3.4.5 INICIO
    private int? idEnEdicion;
    private string nombreEditado = string.Empty;
    private void SeleccionarParaEditar(int id)
    {
        var producto = productos.First(p => p.Id == id);
        idEnEdicion = id;
        seleccion = producto.Nombre;
        nombreEditado = producto.Nombre;
    }

    private void GuardarCambio()
    {
        if (idEnEdicion is null || string.IsNullOrWhiteSpace(nombreEditado))
            return;

        var indice = productos.FindIndex(p => p.Id == idEnEdicion.Value);
        if (indice < 0)
            return;

        productos[indice] = productos[indice] with { Nombre =
nombreEditado.Trim() };
        seleccion = productos[indice].Nombre;
    }
    PASO 3.4.5 FIN */

    /* RETO INTEGRADOR 3.4.6 INICIO
    private string busqueda = string.Empty;
    private decimal? precioMaximo;
    private bool ordenAscendente = true;

    private IEnumerable<Producto> ProductosVisibles =>
        (ordenAscendente
            ? productos.OrderBy(p => p.Precio).ThenBy(p => p.Id)
            : productos.OrderByDescending(p => p.Precio).ThenBy(p => p.Id))
        .Where(p =>
            (string.IsNullOrWhiteSpace(busqueda) ||
             p.Nombre.Contains(busqueda.Trim(),
StringComparison.OrdinalIgnoreCase)) &&
            (precioMaximo is null || p.Precio <= precioMaximo.Value));

    private void AlternarOrden() => ordenAscendente = !ordenAscendente;
    RETO INTEGRADOR 3.4.6 FIN */
```

```
    /* RETO 3.4 INICIO
    private void Limpiar() => seleccion = "ninguno";
    RETO 3.4 FIN */
}
```

Observa tres decisiones. Primero, la página sigue usando TarjetaProducto y el mismo objeto Producto creado en P3.2. Segundo, la variable productos es la única colección fuente; el filtro opcional construye una representación calculada sin borrar objetos. Tercero, el código que todavía no has estudiado se entrega comentado con marcas PASO o RETO, para que puedas activarlo en el momento adecuado.

3.4.3 — Primer cambio: seleccionar un producto mediante un evento

Localiza la etiqueta de botón comentada con PASO 3.4.2. Actívala quitando únicamente las marcas de comentario Razor. Activa también el bloque C# del mismo identificador retirando los delimitadores de comentario que rodean Seleccionar(int id). Guarda, compila y abre la página. Selecciona Lápiz: el texto debe mostrar «Seleccionado: Lápiz» y continuar viendo tres instancias de TarjetaProducto.

La llamada al método está escrita con una lambda porque necesitamos transmitir producto.Id:

```
<button @onclick="() => Seleccionar(producto.Id)">Seleccionar</button>
```

Seleccionar busca en la lista por identidad. No modifica la colección, sino el campo seleccion. Pulsa Cuaderno y comprueba que la selección cambia sin añadir ni borrar tarjetas. Si obtienes un error de compilación que dice que Seleccionar no existe, el botón se ha activado antes que su método C#: ambas piezas forman una única operación.

3.4.4 — Añadir entrada vinculada y alta en memoria

Activa el campo del bloque PASO 3.4.3 y comprueba que su atributo @bind apunta a nuevoNombre. Después activa el bloque PASO 3.4.4, que contiene siguienteId y Agregar; descomenta el botón id="agregar". El método utiliza

string.IsNullOrWhiteSpace para rechazar una entrada vacía, Trim para normalizar el nombre y un Id creciente para crear otro producto.

Introduce Regla, pulsa Añadir producto y cuenta las tarjetas: deben ser cuatro, cada una representada por TarjetaProducto. Intenta añadir una cadena de espacios y verifica que no aparece otra tarjeta. El campo debe quedar vacío tras un alta correcta. Escribe ahora por qué el método de alta pertenece al padre y no al componente TarjetaProducto: el hijo conoce un producto, pero no la colección donde insertar otro.

3.4.5 — Edición por identidad, conservando precio y total

Activa el bloque C# marcado PASO 3.4.5, que define idEnEdicion, nombreEditado, SeleccionarParaEditar y GuardarCambio. La versión de Seleccionar del paso anterior ya no es suficiente para saber qué objeto editar. Por ello deja comentado el botón anterior y activa la versión SeleccionarParaEditar en el mismo contenedor, manteniendo **un solo botón de selección activo por producto**.

Activa también el campo de texto id="editar-nombre" y el botón id="guardar-cambio". Selecciona Lápiz; observa que el editor recibe ese nombre y cambia el texto por «Lápiz rojo». Pulsa Guardar nombre. La tarjeta actualiza su nombre sin perder el Id 2, conserva el precio 1,20 € y el catálogo continúa mostrando tres objetos. Un nombre que contiene solo espacios no debe alterar el dato existente. Explica por qué el operador with sustituye un record sin crear un alta.

3.4.6 — Consulta reversible: búsqueda, límite numérico y ordenación

Este reto integrador añade tres criterios de representación en la misma página. Activa el bloque C# RETO INTEGRADOR 3.4.6 y los cuatro elementos Razor comentados: buscar, precio máximo, botón para alternar orden y resumen de resultados. Por último, cambia la expresión foreach que recorre productos para que recorra ProductosVisibles. **No escribas un segundo foreach ni reemplaces la tarjeta reutilizable.**

Confirma el resultado previsto: orden ascendente Lápiz–Carpeta–Cuaderno, búsqueda «láp» que muestra solo Lápiz, precio máximo 1 que no deja

resultados y precio máximo 2 que permite volver a ver ese producto. Borra los criterios: deben reaparecer todos. Renombra después Lápiz como «Lápiz rojo» y busca «rojo»: el filtro debe encontrar el record editado. Si al borrar el filtro desaparecen definitivamente objetos, has modificado la lista fuente en lugar de producir una consulta.

El input textual utiliza @bind:event="oninput" y puede reaccionar mientras se escribe; el límite decimal conserva el evento por defecto. Esas dos alternativas permiten comparar la semántica del enlace sin introducir JavaScript. La secuencia de consulta no es un requisito para arrancar M03, pero **las instantáneas posteriores mantienen el comportamiento** porque forman parte de la evolución integrada de este catálogo.

3.4.7 — Cierre del punto y transferencia al siguiente

Activa el botón del reto de limpiar selección y comprueba que no modifica ninguna tarjeta. Recupera siempre el código conocido si has provocado un error de manera deliberada. Al finalizar, la aplicación contiene el modelo, la ruta, el componente reutilizable, selección, alta, edición y consulta. No hay una API ni base de datos, porque aún estudiamos estado de componente.

El paso P3.5 conservará estos métodos y la lista. La transformación esencial será mover la intención de selección al hijo y comunicarla mediante EventCallback sin deshacer lo aprendido en P3.3 y P3.4.

Laboratorio de profundización P3.4 — Distinguir entrada, evento y representación

El recorrido principal ya ha permitido seleccionar productos, escribir un nombre y añadirlo a la lista en memoria. Esta actividad opcional está pensada para quien necesita entender por qué cada operación funciona y qué hacer cuando una parte falla. Se ejecuta sobre la carpeta `PROYECTO/3.4/CursoWeb.Blazor`, después de activar los pasos 3.4.2–3.4.4 y, si lo deseas, el reto de limpiar selección. No crees un servicio ni otra página: todos los fenómenos pueden observarse en el componente existente.

Ensayo A: un evento modifica una cadena, no la colección

Con los tres productos iniciales representados, selecciona Cuaderno. Comprueba que el texto cambia a «Seleccionado: Cuaderno» y cuenta las tarjetas. Deben continuar siendo tres. Selecciona después Lápiz y comprueba que el número de tarjetas tampoco cambia. Con este experimento podemos demostrar que la operación `Seleccionar(int id)` solo lee la colección y asigna un nuevo valor a `seleccion`.

Abre `Components/Pages/Productos.razor` y localiza el método. ¿Hay algún `productos.Add` o `productos.RemoveAll` dentro de `Seleccionar`? No. Por tanto, un cambio en el contador de tarjetas indicaría otra modificación adicional o una versión de código distinta de la prevista. El diagnóstico correcto consiste en verificar el método y la colección representada, no en asumir que Blazor modifica automáticamente todos los campos cuando cambia uno.

Resultado razonado: el mismo componente puede mantener varios datos independientes. Un evento debe producir los efectos que define su método; la nueva representación solo refleja el estado resultante. Esta explicación servirá para comprender por qué, en P3.5, eliminar una tarjeta también necesita decidir qué hacer con la selección actual.

Ensayo B: el campo visual y el campo de C# están enlazados

Localiza el input con `id="nombre-nuevo"` y escribe «Marcador». Antes de pulsar Añadir, abandona el campo para confirmar el evento de cambio del enlace. Con un punto de interrupción en el código puedes inspeccionar el valor de `nuevoNombre` cuando se ejecuta `Agregar`. Debería contener el texto introducido.

No confundas el contenido del DOM con el miembro C# si la aplicación no está conectada interactivamente. El atributo `@bind` participa en mantener esa relación, pero también necesita el modo interactivo para coordinar los cambios del componente en este escenario. Si el botón Añadir recibe un texto anterior, investiga cuándo se dispara el evento de enlace y si la entrada cambió realmente antes del clic.

Como ampliación conceptual, consulta la documentación de `@bind:event="oninput"`. No es obligatorio modificar el proyecto, pero debes saber que el momento de

actualización puede ser configurable y que eso afecta a lo que observa el método en ciertos patrones de interacción.

Ensayo C: un nombre inválido no genera una tarjeta

Con el campo vacío, pulsa Añadir. Comprueba que no aparece una tarjeta nueva. Escribe solo espacios y repite. Después escribe «Regla» y verifica que sí se añade. Los tres resultados delimitan la condición `string.IsNullOrWhiteSpace`: no basta con que exista una cadena; debe contener texto significativo después de la comprobación utilizada en este laboratorio.

La prueba no pretende acreditar la misma validación que M02: allí estudiamos DataAnnotations y ModelState para formularios. Aquí estamos aprendiendo cómo un manejador de componente usa un dato enlazado y modifica una colección. El criterio de éxito es observar el efecto real de `Agregar` o su ausencia, no recibir un código HTTP específico.

Solución razonada: una entrada vacía produce un retorno anticipado del método. Una entrada válida alcanza `productos.Add` y después limpia `nuevoNombre`. El control del caso negativo forma parte de la misma función que modifica el estado, de modo que no debemos escribir una excepción de interfaz nueva para este ejercicio.

Ensayo D: el contador local y su identidad

Añade dos productos válidos de manera consecutiva, por ejemplo «Regla» y «Marcador». Comprueba que ambos aparecen y que cada tarjeta recibe un `data-producto-id` distinto. El campo `siguienteId` comienza en 4 y se incrementa después de cada alta. Esto permite distinguir visualmente los objetos durante la vida del componente.

No extrapoles ese contador al ámbito de una base de datos utilizada por miles de usuarios. Aquí no existe concurrencia entre múltiples procesos que comparten un identificador global ni un mecanismo de persistencia. La decisión técnica es adecuada para una lista controlada de laboratorio, y el alumno debe saber explicar esa limitación.

Si los dos objetos aparecen con el mismo Id, comprueba dónde se declara y actualiza `siguienteId`. No debería reinicializarse dentro de `Agregar` en cada clic. Si el nombre aparece repetido, revisa que el campo `nuevoNombre` se limpia después de añadir. La clase de problema determina qué línea merece investigarse.

Ensayo E: el estado visible deja de existir al recargar

Selecciona Lápiz, añade Regla y observa los cambios. Ahora ejecuta una recarga completa de la dirección. Puede reaparecer el catálogo inicial con tres elementos y la selección «ninguno». El resultado se explica por la creación de otra instancia de componente y por la ausencia de almacenamiento externo. No hay ninguna operación de `SaveChangesAsync`.

Compara explícitamente con P2.4: en aquel módulo se verificaba la persistencia después de reiniciar Kestrel y consultar LocalDB; aquí la prueba positiva es el cambio visible en la instancia activa. **Una aplicación puede cumplir correctamente su objetivo sin que todos sus campos sean persistentes**. El problema aparece cuando se confunde el alcance del estado con una promesa de almacenamiento que el diseño nunca realizó.

Ensayo F: dos botones que no deben tener el mismo efecto

Activa el reto de Limpiar. Selecciona Cuaderno y pulsa Limpiar. La selección vuelve a «ninguno» sin reducir el catálogo. Selecciona Lápiz y pulsa Añadir con un nombre válido: la cantidad aumenta, pero la selección continúa en Lápiz salvo que el propio código la cambie. Esta diferencia demuestra que `Limpiar` y `Agregar` operan sobre campos distintos.

Una solución que limpiara automáticamente la selección en todos los eventos podría ser aceptable como otra decisión de diseño, pero no es lo que la práctica especifica. Para verificar reproducibilidad hay que contrastar el comportamiento con el código que realmente se entrega, no con suposiciones sobre lo que debería hacer cualquier interfaz.

Resolver un reporte de error de otro alumno

Supongamos que alguien informa: «Puedo ver las tarjetas, pero el botón Añadir no funciona». No respondas directamente «reinstala Blazor». Pregunta qué

ocurre exactamente: ¿aparece el botón? ¿compila el método? ¿el campo conserva el texto? ¿el clic produce algún cambio? ¿la colección aumenta en el depurador? ¿la página tiene circuito interactivo? Cada respuesta acota una fase distinta.

Si el botón ni siquiera aparece, revisa el comentario Razor. Si existe pero no compila, busca el método `Agregar`. Si compila pero no se ejecuta, revisa modo y conexión. Si ejecuta y retorna, revisa el valor de `nuevoNombre`. Si añade el objeto pero la pantalla no lo representa, revisa el bucle y la actualización del estado. La solución no necesita introducir tecnologías nuevas: requiere comprender el camino completo.

Producto del ejercicio de transferencia

Redacta una explicación de un máximo de una página que siga «escribir Regla → pulsar Añadir → aparecer cuarta tarjeta». Debe nombrar `@bind`, `nuevoNombre`, `@onclick`, `Agregar`, `productos` y el `foreach`, asignando a cada uno una responsabilidad. Contrasta con el recorrido «seleccionar Lápiz → cambiar texto». Si puedes señalar que el segundo no modifica la lista y el primero sí, has comprendido el modelo de estado local de P3.4.

P3.5. Comunicación padre–hijo mediante EventCallback

Laboratorio principal acumulativo P3.5 — El mismo catálogo con comunicación padre–hijo

Objetivo. La aplicación P3.4 ya permite seleccionar, crear, editar, buscar, filtrar y ordenar productos en memoria. En P3.5 **nada de eso desaparece**. El único cambio de responsabilidad obligatorio es que la tarjeta comunica la selección desde un botón propio al padre. Después incorporaremos la intención de quitar y un deshacer de una operación. El padre seguirá siendo dueño del catálogo y del editor.

3.5.1 — Reconocer lo que se conserva del capítulo anterior

Abre M03_BLAZOR/PROYECTO/3.5/CursoWeb.Blazor. Ejecuta la ruta /productos **antes de activar los callbacks**. Comprueba que el listado procede de TarjetaProducto, que puedes renombrar Lápiz, añadir Regla y filtrar los resultados, y que el menú conserva la entrada Productos. El botón Seleccionar todavía está en el padre. Debes poder explicar por qué el componente hijo muestra el nombre y el precio sin modificar la lista por sí mismo.

La diferencia respecto a P3.4 será la ubicación del origen del clic, no el tipo de dato, la ruta ni el almacenamiento. La función SeleccionarParaEditar(int id) seguirá operando sobre la lista de la página. En lugar de llamarla desde un botón externo, el hijo solicitará que el padre la ejecute.

3.5.2 — Estudiar el código completo de la tarjeta

En Components/TarjetaProducto.razor se entrega el siguiente código completo. Lee los comentarios y las propiedades antes de activar nada:

```
<article class="tarjeta" data-producto-id="@Producto.Id">
    <h2>@Producto.Nombre</h2>
    <p>Precio: @Producto.Precio.ToString("0.00") €</p>
    <small>Referencia @Producto.Id</small>
    @* PASO 3.5.2 (ACTIVAR) al declarar AlSeleccionar: *@
    @* <button @onclick="NotificarSeleccion">Seleccionar</button> *@
    @* RETO 3.5 (ACTIVAR) al declarar AlQuitar: *@
    @* <button @onclick="NotificarQuitar">Quitar</button> *@
```

```
</article>
@code {
    [Parameter, EditorRequired] public Producto Producto { get; set; } = default!;

    /* PASO 3.5.2 INICIO
    [Parameter] public EventCallback<int> AlSeleccionar { get; set; }
    private Task NotificarSeleccion() => AlSeleccionar.InvokeAsync(Producto.Id);
    PASO 3.5.2 FIN */

    /* RETO 3.5 INICIO
    [Parameter] public EventCallback<int> AlQuitar { get; set; }
    private Task NotificarQuitar() => AlQuitar.InvokeAsync(Producto.Id);
    RETO 3.5 FIN */
}
```

La entrada Producto está marcada con Parameter y EditorRequired: el padre suministra un Producto concreto. La salida AlSeleccionar, todavía comentada, es EventCallback: la tarjeta envía el Id, no un objeto mutable ni una referencia al contenedor. NotificarSeleccion utiliza InvokeAsync(Producto.Id), que ejecuta el callback vinculado. AlQuitar y NotificarQuitar constituyen la variante posterior de eliminación.

Retira los delimitadores C# de PASO 3.5.2 e incorpora el botón Razor NotificarSeleccion. Una vez hecho esto, el hijo puede emitir una intención, pero aún necesitamos conectar la salida con el método que modifica el estado del padre.

3.5.3 — Entender el archivo completo del padre y cambiar solo el contrato de selección

En Components/Pages/Productos.razor aparece el catálogo heredado, el editor, la consulta y la futura acción de recuperar una eliminación. El código completo del estado inicial del capítulo es:

```
@page "/productos"
@rendermode InteractiveServer
<PageTitle>Catálogo y comunicación</PageTitle>
<h1>P3.5 | El mismo catálogo, ahora con EventCallback</h1>
<p>Conservamos selección, alta, edición y consulta de P3.4. En este punto,
trasladamos el evento de la tarjeta al hijo sin trasladar la propiedad de la
lista.</p>
<p id="seleccion">Seleccionado: @seleccion</p>

<section aria-label="Consultar catálogo">
```

```
    <label>Buscar por nombre: <input id="buscar" @bind="busqueda"
@bind:event="oninput" /></label>
    <label>Precio máximo: <input id="precio-maximo" type="number" min="0"
step="0.5" @bind="precioMaximo" /></label>
    <button id="alternar-orden" @onclick="AlternarOrden">Invertir orden por
precio</button>
    <p id="resumen-filtrado">Mostrando @ProductosVisibles.Count() de
@productos.Count productos</p>
</section>

<div class="tarjetas" id="tarjetas-productos">
    @foreach (var producto in ProductosVisibles)
    {
        <div class="tarjeta-con-acciones" data-elemento-id="@producto.Id">
            @* PASO 3.5.2: pasa la selección desde el botón del padre al callback
del hijo.
               Activa AlSeleccionar en TarjetaProducto y reemplaza la etiqueta
siguiente.
               No mantengas simultáneamente el botón seleccionar-padre. *@
            <TarjetaProducto Producto="producto" />
            @* <TarjetaProducto Producto="producto"
AlSeleccionar="SeleccionarParaEditar" /> *@
            @* RETO 3.5: reemplaza por esta variante tras activar AlQuitar en el
hijo. *@
            @* <TarjetaProducto Producto="producto"
AlSeleccionar="SeleccionarParaEditar" AlQuitar="Quitar" /> *@
            <button class="seleccionar-padre" @onclick="() =>
SeleccionarParaEditar(producto.Id)">Seleccionar</button>
        </div>
    }
</div>
<p id="cantidad">Total: @productos.Count</p>

<section aria-label="Crear y editar productos">
    <label>Nombre nuevo: <input id="nombre-nuevo" @bind="nuevoNombre" /></label>
    <button id="agregar" @onclick="Agregar">Añadir producto</button>
    <label>Editar nombre: <input id="editar-nombre" @bind="nombreEditado" /></
label>
    <button id="guardar-cambio" @onclick="GuardarCambio">Guardar nombre</button>
    <button id="limpiar" @onclick="Limpiar">Limpiar selección</button>
</section>

@* RETO INTEGRADOR 3.5.5: tras conectar AlQuitar a QuitarConDeshacer,
   activa estos controles y el bloque C# de recuperación. *@
@* <button id="deshacer" disabled="@(eliminacionPendiente is null)"
@onclick="DeshacerEliminacion">Deshacer última eliminación</button> *@
@* <p id="estado-deshacer">@TextoDeshacer</p> *@

@code {
    // Estado de P3.4 conservado en P3.5: no se reinicia al cambiar de capítulo.
```

```
    private readonly List<Producto> productos =
    [
        new(1, "Cuaderno", 4.50m),
        new(2, "Lápiz", 1.20m),
        new(3, "Carpeta", 3.75m)
    ];

    private string seleccion = "ninguno";
    private string nuevoNombre = string.Empty;
    private int siguienteId = 4;
    private int? idEnEdicion;
    private string nombreEditado = string.Empty;

    private void SeleccionarParaEditar(int id)
    {
        var producto = productos.First(p => p.Id == id);
        idEnEdicion = id;
        seleccion = producto.Nombre;
        nombreEditado = producto.Nombre;
    }

    private void GuardarCambio()
    {
        if (idEnEdicion is null || string.IsNullOrWhiteSpace(nombreEditado))
            return;
        var indice = productos.FindIndex(p => p.Id == idEnEdicion.Value);
        if (indice < 0)
            return;
        productos[indice] = productos[indice] with { Nombre =
nombreEditado.Trim() };
        seleccion = productos[indice].Nombre;
    }

    private void Agregar()
    {
        if (string.IsNullOrWhiteSpace(nuevoNombre))
            return;
        productos.Add(new Producto(siguienteId++, nuevoNombre.Trim(), 5m));
        nuevoNombre = string.Empty;
    }

    private void Limpiar()
    {
        seleccion = "ninguno";
        idEnEdicion = null;
        nombreEditado = string.Empty;
    }

    private string busqueda = string.Empty;
    private decimal? precioMaximo;
```

```
    private bool ordenAscendente = true;

    private IEnumerable<Producto> ProductosVisibles =>
        (ordenAscendente
            ? productos.OrderBy(p => p.Precio).ThenBy(p => p.Id)
            : productos.OrderByDescending(p => p.Precio).ThenBy(p => p.Id))
        .Where(p =>
            (string.IsNullOrWhiteSpace(busqueda) ||
             p.Nombre.Contains(busqueda.Trim(),
StringComparison.OrdinalIgnoreCase)) &&
            (precioMaximo is null || p.Precio <= precioMaximo.Value));

    private void AlternarOrden() => ordenAscendente = !ordenAscendente;

    /* RETO 3.5 INICIO
    private void Quitar(int id)
    {
        productos.RemoveAll(p => p.Id == id);
        Limpiar();
    }
    RETO 3.5 FIN */

    /* RETO INTEGRADOR 3.5.5 INICIO
    private (Producto Producto, int Indice)? eliminacionPendiente;

    private string TextoDeshacer => eliminacionPendiente is { } pendiente
        ? $"Pendiente: {pendiente.Producto.Nombre}"
        : "Ninguna eliminación pendiente";

    private void QuitarConDeshacer(int id)
    {
        var indice = productos.FindIndex(p => p.Id == id);
        if (indice < 0)
            return;
        eliminacionPendiente = (productos[indice], indice);
        productos.RemoveAt(indice);
        Limpiar();
    }

    private void DeshacerEliminacion()
    {
        if (eliminacionPendiente is not { } anterior)
            return;
        if (!productos.Any(p => p.Id == anterior.Producto.Id))
            productos.Insert(Math.Min(anterior.Indice, productos.Count),
anterior.Producto);
        eliminacionPendiente = null;
    }
    RETO INTEGRADOR 3.5.5 FIN */
}
```

En el bucle ProductosVisibles se crea TarjetaProducto y a su lado un botón con la clase seleccionar-padre. **La transición de este paso consiste en sustituir la etiqueta sin callback por la versión con AlSeleccionar y comentar o eliminar el botón externo**. El resultado debe dejar exactamente una tarjeta reutilizable y un único botón Seleccionar por producto, ahora dentro del hijo.

El padre vincula el callback al método que ya utilizaba el editor:

```
<TarjetaProducto Producto="producto" AlSeleccionar="SeleccionarParaEditar" />
```

No añadas otra definición de SeleccionarParaEditar. Conserva la ya existente, que rellena idEnEdicion, seleccion y nombreEditado, porque esa es la ventaja del cambio: el padre es dueño del estado mientras la tarjeta se convierte en origen del clic.

Prueba positiva: pulsa Seleccionar en Lápiz; el texto del padre cambia y el editor recibe su nombre. Renombra a «Lápiz azul» y guarda. La tarjeta de Id 2 cambia de nombre sin perder precio. **Prueba de continuidad:** añade otro producto con Nombre nuevo, filtra por un fragmento del nuevo nombre e invierte el orden. Son acciones de P3.4 y deben continuar disponibles en P3.5.

3.5.4 — Incorporar eliminación sin modificar directamente la lista desde el hijo

Activa en TarjetaProducto el bloque C# RETO 3.5 y su botón NotificarQuitar. En la página padre activa también el método Quitar del bloque RETO 3.5 y sustituye la variante del componente por otra que conecte ambos callbacks:

```
<TarjetaProducto Producto="producto"
                 AlSeleccionar="SeleccionarParaEditar"
                 AlQuitar="Quitar" />
```

El cuerpo de Quitar busca por Id, retira el elemento de productos y limpia el editor para no dejar seleccionado un objeto que acaba de desaparecer. El hijo **no** recibe toda la lista ni llama a RemoveAll. Pulsa Quitar en Lápiz: el total debe bajar de tres a dos y el subconjunto representado debe reconstruirse. Después

añade Regla y verifica que las tarjetas restantes, el alta y los filtros siguen operando correctamente.

3.5.5 — Recuperar la última eliminación en su posición exacta

Localiza RETO INTEGRADOR 3.5.5 en el padre. Activa el bloque C# completo, que contiene eliminacionPendiente, TextoDeshacer, QuitarConDeshacer y DeshacerEliminacion. En la misma página descomenta el botón id="deshacer" y el texto id="estado-deshacer". Cambia **solo el manejador AlQuitar** para que apunte a QuitarConDeshacer en lugar de Quitar:

```
<TarjetaProducto Producto="producto"
                 AlSeleccionar="SeleccionarParaEditar"
                 AlQuitar="QuitarConDeshacer" />
```

El padre memoriza una tupla con el objeto y el índice que ocupaba, por lo que no tiene que construir un Producto de sustitución. Para recuperar ejecuta Insert sobre el índice guardado (siempre dentro del rango) y descarta la eliminación pendiente. No aparece un historial ilimitado: un segundo borrado antes de deshacer sobrescribe el registro anterior.

Prueba este recorrido completo: inicialmente Lápiz–Carpeta–Cuaderno, con botón Deshacer deshabilitado; elimina Lápiz; observa dos tarjetas y el texto pendiente; pulsa Deshacer; recupera Lápiz–Carpeta–Cuaderno y el Id 2. Después modifica el nombre del producto recuperado y confirma que el editor y el buscador continúan funcionando, igual que antes de añadir callbacks.

3.5.6 — Diagnosticar comportamientos sin simplificar el proyecto

Si aparecen dos botones Seleccionar, el alumno ha mantenido el botón externo y el nuevo botón del hijo. Si aparecen seis tarjetas, ha dejado dos variantes de TarjetaProducto activas. Si al eliminar una tarjeta los filtros desaparecen, ha sustituido el componente completo por un ejemplo mínimo en lugar de evolucionar el existente. Si deshacer coloca Lápiz al final, ha utilizado Add en lugar de Insert; si lo recupera dos veces, no ha limpiado eliminacionPendiente.

Entrega del punto: una página con la misma ruta, modelo, navegación, tarjetas, editor y consulta de P3.4, ahora con callbacks de selección y

eliminación y una recuperación local reversible. Este estado será el punto de partida de P3.6. Los resultados no se guardan todavía en base de datos.

Laboratorio de profundización P3.5 — Dos contratos y un dueño del estado

Esta actividad opcional aprovecha el proyecto 3.5 con los bloques de selección y reto ya activados. No introduce ningún servicio ni endpoint adicional. Su finalidad es comprobar que la comunicación entre componentes es realmente explícita, tipada y coherente con la propiedad del estado.

Primer recorrido: entrada del padre hacia la tarjeta

En `Productos.razor` encuentra el `foreach` y la etiqueta `TarjetaProducto` que recibe `Producto="producto"`. Abre el hijo y localiza su propiedad pública marcada con `[Parameter]`. Escribe una explicación causal: cada iteración del padre suministra un objeto concreto y el hijo muestra nombre, precio e Id de ese objeto.

La tarjeta no posee la colección completa. Si decide mostrar únicamente los campos del producto recibido, puede reutilizarse donde haya otra colección semejante. Por ello conviene distinguir el **dato de entrada** del contrato de evento que utilizaremos después.

Caso negativo reversible: sustituir `Producto="producto"` por `Producto="productos[0]"` produciría varias tarjetas que muestran el primer producto. No es un problema de EventCallback ni de navegación, sino de información equivocada transmitida en el punto de composición. Restablece la expresión correcta si realizas el experimento.

Segundo recorrido: salida del hijo hacia el padre

Con los callbacks activos, pulsa Seleccionar en Lápiz. La tarjeta conoce `Producto.Id` y ejecuta `NotificarSeleccion`. Ese método invoca `AlSeleccionar.InvokeAsync(Producto.Id)`. El padre enlaza la propiedad `AlSeleccionar` con su método `Seleccionar(int)` y actualiza el campo `seleccion`. La tarjeta no necesita cambiar el texto del padre por acceso directo; comunica un Id para que el padre decida.

El elemento visible `id="seleccion"` está fuera de la tarjeta, lo que hace observable la separación: el evento se origina dentro del hijo pero su efecto pertenece al estado del padre. Un sistema basado en referencias ocultas al componente contenedor sería más difícil de comprender y reutilizar.

Tercer recorrido: una solicitud de eliminación

Pulsa Quitar en la tarjeta de Lápiz. El hijo invoca `AlQuitar` con el mismo Id, y el padre elimina de su lista los objetos que coincidan. Debes observar tres efectos: la tarjeta desaparece, el total disminuye y la selección vuelve a «ninguno». Los tres dependen de que el padre actualice correctamente sus campos.

Si la tarjeta desaparece pero el texto sigue diciendo que Lápiz está seleccionado, `EventCallback` probablemente ha funcionado. El defecto se encontraría en el método `Quitar`, porque no ha limpiado `seleccion` como exige la solución del reto. Esta es una enseñanza importante: **comunicación correcta** y **regla de estado coherente** no son sinónimos.

Investigar por qué las tarjetas aparecen duplicadas

El proyecto contiene tres variantes de componente padre para permitir la evolución docente: sin callbacks, con selección y con selección y eliminación. Si dejas activas varias, cada producto puede representarse dos o tres veces. El síntoma es un número incorrecto de elementos HTML, pero la colección C# puede continuar teniendo exactamente tres objetos. No intentes corregirlo eliminando elementos de `productos`: revisa las etiquetas `TarjetaProducto` activas dentro del bucle.

La explicación causal se basa en composición, no en la lógica del callback. Cada etiqueta instancia un componente; dos etiquetas representan dos instancias. El sistema no sabe que una de ellas es «la versión antigua del ejercicio» si el alumno no la ha comentado.

Estudiar un tipo de callback incorrecto

El contrato `EventCallback<int>` expresa que el hijo notificará un entero. ¿Qué ocurriría si intentáramos pasar una cadena o un objeto no compatible con ese

contrato? El compilador o el contrato de invocación ya no sería coherente. No es necesario alterar la práctica hasta dejarla rota; basta identificar en ambos archivos dónde se declara el tipo y qué valor se envía.

Esa disciplina de tipos aporta una ventaja importante: el padre puede diseñar su método receptor sabiendo qué clase de dato recibirá. Sin embargo, no hay que atribuirle validación de identidad, autorización o persistencia; un entero bien tipado podría no pertenecer a un producto de la lista, y una operación autorizada exigiría requisitos que todavía no estudiamos en M03.

Una propuesta alternativa que evitaremos

Imagina que la tarjeta almacenara una copia completa de todos los productos y eliminara por sí misma uno de ellos. Habría dos fuentes de estado, la del padre y la del hijo, que podrían evolucionar de forma distinta. Nuestro diseño evita ese problema porque la tarjeta solo conoce el objeto que presenta y los callbacks de acciones posibles. El padre mantiene la colección y decide su actualización.

No hay que aplicar siempre este patrón a cualquier componente del mundo. Un componente puede tener estado local propio, y muchos escenarios no necesitan callbacks. El criterio es que **la responsabilidad de modificar una colección debe quedar en el lugar que la posee o coordina**, mientras los hijos comunican intenciones mediante contratos claros.

Construir una evidencia visual de cada contrato

Con el navegador abierto, anota una prueba que demuestre cada dirección. Para padre→hijo, compara los valores de dos tarjetas diferentes. Para hijo→padre, pulsa Seleccionar y observa el texto externo. Para el reto de Quitar, verifica reducción del total. La compilación por sí sola no demuestra que la notificación llegue; el clic real sí permite observarlo.

No te conformes con una captura final que muestre solo dos productos: podría proceder de una colección inicial con dos elementos. La evidencia más convincente contiene la transición desde tres a dos causada por un botón concreto. Esto es especialmente relevante al ejecutar pruebas en navegador.

Una transferencia hacia M04 sin implementarla aún

Supongamos que en el módulo siguiente el padre dispone de un servicio HTTP para eliminar o actualizar un producto. La tarjeta podría seguir emitiendo el Id; el padre decidiría si invoca el servicio, muestra un error o refresca la lista. Esa posibilidad deriva de mantener el evento desacoplado del transporte. No añadas HTTP a P3.5: la finalidad es comprender el contrato antes de construir una operación remota.

Conclusión del laboratorio

Redacta en tu cuaderno dos frases distintas: «El padre entrega el producto por [Parameter]» y «El hijo notifica una intención por EventCallback». Después añade una tercera: «El padre aplica la modificación a su colección». Si las tres describen el comportamiento observado y puedes localizar las líneas correspondientes en los archivos, has comprendido la comunicación padre–hijo de este módulo.

P3.6. OnInitializedAsync, servicio en memoria y estados de carga

Laboratorio principal acumulativo P3.6 — Los datos se cargan de forma asíncrona sin perder el editor

La aplicación del capítulo anterior ya contiene un catálogo reutilizable, selección por EventCallback, alta, edición, filtros, ordenación, eliminación y deshacer. No vamos a sustituirlo por una página que **solo** sabe mostrar una lista. El nuevo aprendizaje es obtener los **datos iniciales** desde un servicio en memoria, representar la espera y distinguir un resultado vacío de un fallo conocido.

3.6.1 — Localizar las cinco piezas que intervienen

El trabajo sigue en la ruta /productos del proyecto M03_BLAZOR/PROYECTO/3.6/CursoWeb.Blazor. Abre Program.cs, Services/IProductosMemoriaService.cs, Services/ProductosMemoriaService.cs, Components/TarjetaProducto.razor y Components/Pages/Productos.razor. Los dos últimos contienen todavía el catálogo y los callbacks del capítulo anterior; los tres primeros introducen el contrato, la implementación y el registro del servicio.

El archivo Program.cs completo del estado de este ejercicio es:

```
using CursoWeb.Blazor.Components;
using CursoWeb.Blazor.Services;

var builder = WebApplication.CreateBuilder(args);
// Servicios necesarios para componentes Razor y circuitos Interactive Server.
builder.Services.AddRazorComponents().AddInteractiveServerComponents();
// PASO 3.6.1: inyección por interfaz. No existe HttpClient ni API aún.
builder.Services.AddScoped<IProductosMemoriaService, ProductosMemoriaService>();
var app = builder.Build();
if (!app.Environment.IsDevelopment()) app.UseExceptionHandler("/error");
app.UseStaticFiles();
app.UseAntiforgery();
// El modo Interactive Server necesita tanto servicios como endpoint.
app.MapRazorComponents<App>().AddInteractiveServerRenderMode();
app.Run();
public partial class Program { }
```

La llamada AddScoped registra la implementación detrás de la interfaz; la página puede solicitar IProductosMemoriaService mediante @inject. El resto del host no se descarta: conserva los componentes Razor, Interactive Server, antiforgery y el mapeo de endpoints que ya se introdujeron en 3.1.

El contrato completo del servicio, IProductosMemoriaService.cs, se entrega así:

```
using CursoWeb.Blazor.Models;
namespace CursoWeb.Blazor.Services;
public interface IProductosMemoriaService
{
    Task<IReadOnlyList<Producto>> ObtenerAsync();
    /* RETO INTEGRADOR 3.6.7 INICIO
    Task<IReadOnlyList<Producto>> ObtenerConErrorAsync(bool simularError);
    RETO INTEGRADOR 3.6.7 FIN */
}
```

La interfaz promete obtener una lista tipada de forma asíncrona. El contrato no prescribe si el origen es una base de datos o un array local; la implementación de este módulo no utiliza red ni EF Core. El segundo método está preparado como reto de error y reintento, pero permanece comentado hasta el paso correspondiente.

La clase ProductosMemoriaService.cs implementa ese contrato y devuelve tres productos tras una pequeña espera:

```
using CursoWeb.Blazor.Models;
namespace CursoWeb.Blazor.Services;
/// <summary>Servicio de laboratorio sin HTTP, EF Core ni base externa.</summary>
public sealed class ProductosMemoriaService : IProductosMemoriaService
{
    public async Task<IReadOnlyList<Producto>> ObtenerAsync()
    {
        await Task.Delay(700);
        // RETO 3.6 (alternativa): sustituye el bloque return siguiente por:
        // return Array.Empty<Producto>();
        return
        [
            new Producto(1, "Cuaderno", 4.50m),
            new Producto(2, "Lápiz", 1.20m),
            new Producto(3, "Carpeta", 3.75m)
        ];
    }
    /* RETO INTEGRADOR 3.6.7 INICIO
```

```
    public async Task<IReadOnlyList<Producto>> ObtenerConErrorAsync(bool
simularError)
    {
        await Task.Delay(350);
        if (simularError)
            throw new InvalidOperationException("Fallo controlado del servicio
local.");
        return await ObtenerAsync();
    }
    RETO INTEGRADOR 3.6.7 FIN */

}
```

Task.Delay es una simulación temporal para hacer visible el estado de carga. No representa un requisito de producción. Cuando el método devuelve Array.Empty() en la alternativa indicada, es un **éxito sin elementos** y no una excepción.

3.6.2 — Confirmar que conservamos TarjetaProducto y EventCallback

El componente de tarjeta continúa completo, incluyendo los dos contratos de notificación que aprendimos en P3.5:

```
<article class="tarjeta" data-producto-id="@Producto.Id">
    <h2>@Producto.Nombre</h2>
    <p>Precio: @Producto.Precio.ToString("0.00") €</p>
    <small>Referencia @Producto.Id</small>
    <button @onclick="NotificarSeleccion">Seleccionar</button>
    <button @onclick="NotificarQuitar">Quitar</button>
</article>
@code {
    [Parameter, EditorRequired] public Producto Producto { get; set; } = default!;
    [Parameter] public EventCallback<int> AlSeleccionar { get; set; }
    [Parameter] public EventCallback<int> AlQuitar { get; set; }
    private Task NotificarSeleccion() => AlSeleccionar.InvokeAsync(Producto.Id);
    private Task NotificarQuitar() => AlQuitar.InvokeAsync(Producto.Id);
}
```

AlSeleccionar y AlQuitar comunican intenciones por Id. La carga asíncrona no altera la responsabilidad del hijo. El padre continuará gestionando todos los cambios de los objetos, porque posee la colección cargada.

3.6.3 — Estudiar el archivo completo de la página y activar la carga inicial

El código completo del estado inicial entregado para P3.6 está a continuación. Su longitud es necesaria porque **incluye los comportamientos acumulados**, no porque hayamos creado otro catálogo reducido.

```
@page "/productos"
@rendermode @(new InteractiveServerRenderMode(prerender: false))
@inject IProductosMemoriaService Catalogo
<PageTitle>Catálogo asíncrono</PageTitle>
<h1>P3.6 | Catálogo acumulativo y OnInitializedAsync</h1>
<p>Conservamos la selección, el alta, la edición, los filtros y la recuperación de
P3.5. Ahora los datos iniciales proceden de un servicio.</p>
<p id="estado-carga">@estado</p>
<p id="seleccion">Seleccionado: @seleccion</p>
@* RETO INTEGRADOR 3.6.7: error controlado y botón de reintento.
   Activa junto con la nueva versión de OnInitializedAsync y el servicio. *@
@* <label>Simular fallo: <input id="simular-error" type="checkbox"
@bind="simularError" /></label> *@
@* <button id="recargar-productos" disabled="@cargando"
@onclick="RecargarAsync">Recargar catálogo</button> *@
@* @if (!string.IsNullOrEmpty(errorCarga)) { <p role="alert" id="error-
carga">@errorCarga</p> } *@
@if (!cargando)
{
    <section aria-label="Consultar catálogo">
        <label>Buscar por nombre: <input id="buscar" @bind="busqueda"
@bind:event="oninput" /></label>
        <label>Precio máximo: <input id="precio-maximo" type="number" min="0"
step="0.5" @bind="precioMaximo" /></label>
        <button id="alternar-orden" @onclick="AlternarOrden">Invertir orden por
precio</button>
        <p id="resumen-filtrado">Mostrando @ProductosVisibles.Count() de
@productos.Count productos</p>
    </section>

    @if (productos.Count > 0)
    {
        <div class="tarjetas" id="tarjetas-productos">
            @foreach (var producto in ProductosVisibles)
            {
                <TarjetaProducto Producto="producto"
                                 AlSeleccionar="SeleccionarParaEditar"
                                 AlQuitar="QuitarConDeshacer" />
            }
        </div>
    }
    <p id="cantidad">Total: @productos.Count</p>
```

```
    @* RETO 3.6: condición para una colección vacía tras una carga correcta. *@
    @* @if (productos.Count == 0 && estado == "Lista preparada") { <p id="sin-
productos">No hay productos para mostrar.</p> } *@

    <section aria-label="Crear y editar productos">
        <label>Nombre nuevo: <input id="nombre-nuevo" @bind="nuevoNombre" /></
label>
        <button id="agregar" @onclick="Agregar">Añadir producto</button>
        <label>Editar nombre: <input id="editar-nombre" @bind="nombreEditado" /></
label>
        <button id="guardar-cambio" @onclick="GuardarCambio">Guardar nombre</
button>
        <button id="limpiar" @onclick="Limpiar">Limpiar selección</button>
    </section>

    <button id="deshacer" disabled="@(eliminacionPendiente is null)"
@onclick="DeshacerEliminacion">Deshacer última eliminación</button>
    <p id="estado-deshacer">@TextoDeshacer</p>
}

@code {
    // P3.6: se deja vacía al empezar; el servicio cargará el catálogo inicial.
    private List<Producto> productos = [];
    private bool cargando = true;
    private string estado = "Servicio pendiente de activar";

    // P3.4: estado del editor, el alta y los filtros, conservado.
    private string seleccion = "ninguno";
    private string nuevoNombre = string.Empty;
    private int siguienteId = 4;
    private int? idEnEdicion;
    private string nombreEditado = string.Empty;
    private string busqueda = string.Empty;
    private decimal? precioMaximo;
    private bool ordenAscendente = true;

    private IEnumerable<Producto> ProductosVisibles =>
        (ordenAscendente
            ? productos.OrderBy(p => p.Precio).ThenBy(p => p.Id)
            : productos.OrderByDescending(p => p.Precio).ThenBy(p => p.Id))
        .Where(p =>
            (string.IsNullOrWhiteSpace(busqueda) ||
             p.Nombre.Contains(busqueda.Trim(),
StringComparison.OrdinalIgnoreCase)) &&
            (precioMaximo is null || p.Precio <= precioMaximo.Value));

    private void AlternarOrden() => ordenAscendente = !ordenAscendente;

    private void SeleccionarParaEditar(int id)
```

```
    {
        var producto = productos.First(p => p.Id == id);
        idEnEdicion = id;
        seleccion = producto.Nombre;
        nombreEditado = producto.Nombre;
    }

    private void GuardarCambio()
    {
        if (idEnEdicion is null || string.IsNullOrWhiteSpace(nombreEditado))
            return;
        var indice = productos.FindIndex(p => p.Id == idEnEdicion.Value);
        if (indice < 0)
            return;
        productos[indice] = productos[indice] with { Nombre =
nombreEditado.Trim() };
        seleccion = productos[indice].Nombre;
    }

    private void Agregar()
    {
        if (string.IsNullOrWhiteSpace(nuevoNombre))
            return;
        productos.Add(new Producto(siguienteId++, nuevoNombre.Trim(), 5m));
        nuevoNombre = string.Empty;
    }

    private void Limpiar()
    {
        seleccion = "ninguno";
        idEnEdicion = null;
        nombreEditado = string.Empty;
    }

    // P3.5: la tarjeta comunica intención; el padre gestiona la lista y el
deshacer.
    private (Producto Producto, int Indice)? eliminacionPendiente;
    private string TextoDeshacer => eliminacionPendiente is { } pendiente
        ? $"Pendiente: {pendiente.Producto.Nombre}"
        : "Ninguna eliminación pendiente";

    private void QuitarConDeshacer(int id)
    {
        var indice = productos.FindIndex(p => p.Id == id);
        if (indice < 0)
            return;
        eliminacionPendiente = (productos[indice], indice);
        productos.RemoveAt(indice);
        Limpiar();
    }
```

```
    private void DeshacerEliminacion()
    {
        if (eliminacionPendiente is not { } anterior)
            return;
        if (!productos.Any(p => p.Id == anterior.Producto.Id))
            productos.Insert(Math.Min(anterior.Indice, productos.Count),
anterior.Producto);
        eliminacionPendiente = null;
    }

    /* PASO 3.6.2 INICIO
    protected override async Task OnInitializedAsync()
    {
        estado = "Cargando productos...";
        productos = (await Catalogo.ObtenerAsync()).ToList();
        cargando = false;
        estado = "Lista preparada";
    }
    PASO 3.6.2 FIN */

    /* RETO INTEGRADOR 3.6.7 INICIO
    private bool simularError;
    private string? errorCarga;

    // Sustituye el método de PASO 3.6.2 por esta versión al llegar al reto.
    protected override Task OnInitializedAsync() => RecargarAsync();

    private async Task RecargarAsync()
    {
        cargando = true;
        errorCarga = null;
        estado = "Cargando productos...";
        productos = [];
        Limpiar();
        eliminacionPendiente = null;
        try
        {
            productos = (await
Catalogo.ObtenerConErrorAsync(simularError)).ToList();
            siguienteId = Math.Max(4, productos.Select(p =>
p.Id).DefaultIfEmpty(3).Max() + 1);
            estado = "Lista preparada";
        }
        catch (InvalidOperationException)
        {
            errorCarga = "No se pudo cargar el catálogo de demostración.";
            estado = "Carga fallida";
        }
        finally
```

```
        {
            cargando = false;
        }
    }
    RETO INTEGRADOR 3.6.7 FIN */
}
```

En el comienzo aparece @inject IProductosMemoriaService Catalogo. La directiva de renderizado utiliza InteractiveServerRenderMode(prerender: false) para observar de forma controlada la carga en el circuito. En @code, productos empieza como List vacía; no contiene por segunda vez los tres productos literales. El resto del estado conserva el editor, las variables de filtro y los métodos de eliminación del punto anterior.

Localiza PASO 3.6.2 INICIO y FIN en la parte final del archivo. Retira los dos delimitadores de comentario C#, dejando activo OnInitializedAsync. El método cambia estado a «Cargando productos...», espera Catalogo.ObtenerAsync, convierte el resultado a List para que el padre pueda añadir/quitar/editar y finalmente desactiva cargando y muestra «Lista preparada».

Arranca el proyecto en Kestrel y abre /productos: en unos instantes deben aparecer tres tarjetas. A diferencia de P3.5, los objetos ya no se inicializan con un literal en el componente. Mientras cargando es true, los controles operativos se ocultan; cuando la carga finaliza, reaparecen las mismas funciones de búsqueda, alta y edición, con los callbacks del componente hijo.

3.6.4 — Confirmar la continuidad funcional después del await

Esta comprobación es el corazón del ejercicio. Selecciona Lápiz usando el botón **del componente hijo**, cambia su nombre por «Lápiz recuperado», guarda y búscalo por «recuperado». Borra la búsqueda y añade Regla: el catálogo debe tener cuatro tarjetas. Quita Lápiz recuperado y utiliza Deshacer. Comprueba que se recupera el Id y el nombre editado. Alterna la ordenación; los datos no deben perderse.

Si cualquiera de esas operaciones desaparece en P3.6, el proyecto no es una evolución válida del capítulo anterior aunque el método OnInitializedAsync

funcione. Hay que reparar la pérdida de funcionalidad, no eliminar los ejemplos para simplificar el laboratorio.

3.6.5 — Representar una carga correcta sin elementos

El marcado incluye una condición comentada con RETO 3.6 para mostrar el párrafo id="sin-productos" cuando la carga ha terminado y la lista es vacía. Actívala retirando sus delimitadores Razor. Cambia temporalmente el retorno de ObtenerAsync por Array.Empty() y recarga el navegador. Tras la espera, aparece «Lista preparada» y el mensaje de lista vacía. Esto demuestra que terminar una operación sin elementos **no es lo mismo** que entrar en catch.

Restaura después el retorno original de tres productos antes del reto siguiente. No mantengas dos return consecutivos ni cambies la interfaz por un servicio HTTP inexistente.

3.6.6 — Extender el servicio para un fallo controlado

Activa en la interfaz y el servicio los bloques C# RETO INTEGRADOR 3.6.7. El nuevo método ObtenerConErrorAsync(bool simularError) espera brevemente, lanza InvalidOperationException si simularError es true y, en caso contrario, obtiene la lista normal. El objetivo es poder forzar tanto el error como la recuperación **sin depender de una API externa**.

En Productos.razor activa el bloque C# del mismo reto, pero **sustituye** el OnInitializedAsync anterior por el nuevo que delega en RecargarAsync. Es importante que exista una sola definición activa de OnInitializedAsync. RecargarAsync cambia las banderas y el mensaje de espera, limpia el error previo y vacía la lista visible, intenta cargar, informa de la excepción conocida y desactiva la bandera dentro de finally. Activa también la casilla simular-error, el botón recargar-productos y el párrafo condicional error-carga.

3.6.7 — Fallar, recuperar y volver a utilizar el catálogo

Con el servicio correcto, espera las tres tarjetas. Marca Simular fallo y pulsa Recargar catálogo: debe aparecer «Carga fallida», un aviso textual y ninguna tarjeta del intento anterior. Desmarca la casilla, pulsa de nuevo el botón y observa cómo vuelve «Lista preparada» con los tres productos.

Tras la recuperación, ejecuta **otra vez** las operaciones del apartado 3.6.4: selección desde la tarjeta hija, edición por Id, alta de otro producto, filtros y deshacer. Un resultado de recarga que dibuja tres tarjetas pero deja de permitir editar no cumple el objetivo de este capítulo.

3.6.8 — Explicar qué se ha construido

Identifica para cada funcionalidad el archivo responsable: Program.cs registra servicios; la interfaz define operaciones; ProductosMemoriaService implementa la obtención; Productos.razor conserva lista, filtros, edición y estados de carga; TarjetaProducto recibe Producto y emite callbacks. Relaciona cada mecanismo con los ejemplos de teoría 3.3–3.6. El resultado no es un CRUD remoto: los cambios permanecen en memoria y volver a cargar los datos del servicio reinicia el catálogo local.

Laboratorio de profundización P3.6 — Carga asíncrona, resultado vacío y diagnóstico

El objetivo de esta ampliación es distinguir cuatro hechos que pueden resultar visualmente parecidos: el método de inicialización no está activado, el método está esperando, el servicio terminó con datos y el servicio terminó con una lista vacía. Todos pueden observarse con el mismo proyecto 3.6, sin API remota ni SQL. Trabaja sobre una copia docente limpia si has realizado varias modificaciones.

Caso A: el componente existe, pero no llama al servicio

Abre `/productos` con el bloque `PASO 3.6.2` todavía comentado. Debes leer «Servicio pendiente de activar». Comprueba que `Program.cs` registra correctamente `IProductosMemoriaService` y que el archivo de la página incluye `@inject`. Aunque esas piezas existan, el método `OnInitializedAsync` no se ejecuta porque permanece comentado. La dependencia está preparada, pero ninguna operación de carga ha sido programada en la instancia.

Este caso demuestra que registrar un servicio no hace que todos los componentes lo llamen automáticamente. La llamada real se encuentra en el método de inicialización que activaremos en el siguiente estado. No confundas **disponer de una dependencia** con **invocarla**.

Caso B: el servicio recibe la llamada y la pantalla muestra espera

Activa `OnInitializedAsync` y ejecuta de nuevo. El método asigna `estado = "Cargando productos..."` antes del `await`. Dado que la página utiliza Interactive Server con `prerender: false`, podemos observar el momento en que el componente prepara la espera. El servicio contiene `Task.Delay(700)` como latencia simulada. No utilizamos un temporizador de presentación ni introducimos una segunda tarea innecesaria; la espera está en la operación de obtención de datos.

En el navegador, intenta observar la transición de «Cargando productos…» a «Lista preparada». La duración exacta puede variar, porque compilar y abrir recursos también consume tiempo. El resultado esencial no es medir milisegundos, sino distinguir los estados y comprender qué código los produce.

Caso C: los datos llegan, pero no hay tarjetas

Para reproducir un error de presentación, comenta temporalmente `cargando = false` después de la llamada al servicio. Aunque `productos` reciba la lista, la condición `!cargando && productos.Count > 0` no permite representarla. Esto explica un informe frecuente: «el servicio parece devolver productos, pero la página está vacía». La causa no tiene que ser el transporte; puede encontrarse en una bandera que el componente utiliza para decidir qué dibujar.

Restablece la asignación. Ahora las tarjetas deben aparecer cuando termine la carga. La solución razonada consiste en mantener coherentes los datos obtenidos y el indicador de finalización; no es mover las tarjetas a otro layout ni eliminar la condición para ocultar el problema.

Caso D: el servicio termina correctamente con cero productos

Activa el reto del mensaje `sin-productos` y sustituye temporalmente el retorno del servicio por `Array.Empty<Producto>()`. Conserva la espera para observar la transición. La carga terminará y la lista quedará vacía, pero la interfaz ya no debería quedarse en un estado ambiguo: debe informar «No hay productos para mostrar».

Este escenario es distinto del anterior. En C la condición estaba bloqueada por una bandera que nunca se actualizó; en D la bandera indica finalización y la

colección contiene cero elementos de forma legítima. **Resultado vacío no equivale a fallo del servicio**. El alumno debe demostrarlo observando el estado «Lista preparada» junto al mensaje de ausencia de datos.

Caso E: el servicio no está registrado

Como ejercicio conceptual, imagina que se eliminara `AddScoped<IProductosMemoriaService, ProductosMemoriaService>()` de `Program.cs` mientras el componente sigue solicitando `@inject IProductosMemoriaService Catalogo`. La inyección de dependencias ya no tendría una implementación disponible. El diagnóstico debería dirigirse al registro del contenedor, no a la sintaxis del `foreach` ni a los atributos HTML.

No recomendamos romper el proyecto como actividad obligatoria si el alumno todavía está aprendiendo a cargar datos. Basta con localizar las dos piezas y explicar por qué deben corresponderse. La teoría 3.6 detalla esta relación y su conexión con la inyección utilizada anteriormente en Web API.

Caso F: inicialización duplicada por prerenderizado

En una aplicación Blazor Web App con prerenderizado, `OnInitializedAsync` puede ejecutarse durante la representación inicial en servidor y otra vez cuando se inicia el componente interactivo. Por eso el proyecto 3.6 declara `prerender: false`: permite centrar la práctica en una transición controlada. Esta decisión no debe convertirse en una regla universal para producción, donde el prerenderizado puede ser valioso.

Como transferencia, imagina que el servicio realizara una operación no idempotente en lugar de devolver datos. Dos inicializaciones podrían duplicar efectos. ¿Qué deberíamos investigar? La estrategia de prerenderizado, la ubicación de la operación y cómo conservar o compartir resultados. No necesitas implementar esas soluciones avanzadas en M03; basta comprender por qué sería importante analizar el ciclo de vida.

Resolver un reporte de error paso a paso

Un alumno comunica: «La lista se queda en blanco». Comienza preguntando si hay un mensaje visible. Si dice «Servicio pendiente de activar», revisa que el

bloque de `OnInitializedAsync` esté activo. Si dice «Cargando productos...» durante mucho tiempo, revisa si el método termina y si hay errores durante `ObtenerAsync`. Si dice «Lista preparada» pero no aparecen tarjetas, comprueba `cargando` y `productos.Count`. Si dice «No hay productos para mostrar», quizá la lista esté vacía como resultado válido del servicio o se haya dejado activa la variante del reto.

La observación de estados reduce diagnósticos erróneos. No toda pantalla sin tarjetas requiere un cambio de `@rendermode` o un nuevo servidor HTTP. En este proyecto la red no interviene como origen de datos, así que no debemos inventar un problema de API para explicar una lista vacía.

Respuesta razonada esperada

Elabora una explicación que conecte `@inject`, el registro `AddScoped`, `OnInitializedAsync`, `await Catalogo.ObtenerAsync()`, `cargando` y el marcado condicional. Debes poder señalar el archivo responsable de cada etapa: `Program.cs` configura el contenedor, `IProductosMemoriaService.cs` declara el contrato, `ProductosMemoriaService.cs` suministra datos y `Productos.razor` administra la presentación.

Si puedes diferenciar los resultados sin API, estarás preparado para estudiar M04: allí los mismos estados de interfaz deberán convivir con errores HTTP, deserialización JSON y respuestas remotas. No se trata de repetir desde cero la programación de componentes, sino de cambiar de manera controlada la fuente de datos.

Taller integrador — Construir y explicar el catálogo Blazor de principio a fin

Este taller **no es una séptima aplicación ni un nuevo tema oficial**. Es una oportunidad de reconstruir el itinerario utilizando los archivos completos ya estudiados y de justificar por qué cada componente existe. Se puede realizar por equipos o como actividad posterior a las seis horas principales. El resultado ha de conservar las funciones de todos los capítulos.

Fase A — Dibujar el recorrido antes de escribir código

Recorre la aplicación desde la URL hasta el producto representado. Una solicitud dirigida a /productos encuentra la página por su directiva @page. Esa página se representa a través del router y del layout que ofrece la navegación. La página solicita los datos iniciales a Catalogo después de obtener su dependencia por @inject. Al concluir la tarea asíncrona, recorre una lista de objetos Producto y suministra cada objeto a TarjetaProducto. El componente hijo produce su artículo HTML y comunica intenciones por los callbacks configurados por su padre.

Distingue por escrito las cinco responsabilidades: **arranque**, **enrutamiento**, **obtención de datos**, **estado/operaciones** y **representación/reutilización**. No las sustituyas por el nombre de un único framework: cada responsabilidad se localiza en uno o varios archivos que has leído completos. Program.cs registra el servicio; App.razor y Routes.razor configuran el árbol de presentación; MainLayout.razor organiza navegación; Productos.razor gobierna catálogo, filtros y editor; TarjetaProducto.razor representa un producto; el servicio implementa la adquisición.

Fase B — Reconstruir desde la plantilla sin omitir archivos

Crea un nuevo proyecto Blazor Web App de .NET 8 en un directorio **temporal separado**. Si usas Visual Studio 2026, selecciona la opción de componentes con interactividad en servidor; si empleas la CLI, verifica el template y el target framework para que el resultado use net8.0. No reutilices las clases MVC A de los módulos anteriores, ya que el proyecto Blazor B tiene otro host y otro árbol de componentes.

Antes de añadir la página Productos, abre el proyecto de partida P3.1 y compara sus archivos completos con los que generó la plantilla. Identifica si el archivo Program.cs registra Interactive Server en servicios y en endpoints. Comprueba que App.razor conserva los recursos necesarios para la interactividad y que Routes.razor declara el router. Copia o reproduce MainLayout.razor, Home.razor, Estatica.razor e Interactiva.razor según el código ya explicado en P3.1, manteniendo sus rutas sin nombres duplicados.

Añade también el contenido de app.css. Sin esta hoja, las páginas podrían funcionar funcionalmente pero no presentar el estilo de tarjetas, navegación y mensajes que el alumno espera. Localiza la referencia a la hoja en App.razor y verifica que las clases usadas en el componente tienen reglas de presentación. No introduzcas reglas CSS arbitrarias de una aplicación diferente: el propósito es entender cómo una plantilla sirve recursos estáticos al navegador.

El archivo launchSettings.json tiene otro cometido: su configuración afecta a los perfiles con los que el entorno de desarrollo arranca Kestrel. No sustituye a @page, no registra el servicio y no configura el callback de una tarjeta. Si un alumno cambia el puerto, debe distinguir el perfil de lanzamiento del destino que escribe en el navegador. El proyecto puede ejecutarse con una URL explícita en la CLI, aunque Visual Studio utilice el perfil descrito en el archivo.

Fase C — Ver el catálogo antes de abstraerlo

Reproduce el contenido de Models/Producto.cs y la primera versión de Productos.razor de P3.2. La lista literal de Cuaderno, Lápiz y Carpeta pertenece inicialmente al componente padre. Recorre los tres records y muestra un li por objeto. Añade NavLink hacia /productos en el layout. No cambies simultáneamente el router y la colección: primero comprueba que se puede navegar, después verifica que el listado contiene los tres elementos.

Ahora extrae el marcado visual a TarjetaProducto.razor, como se hizo en P3.3. La página no tiene por qué conocer el HTML exacto que el hijo utiliza, pero sí debe suministrar el parámetro obligatorio Producto en cada vuelta del foreach. Modifica el componente hijo para cambiar, por ejemplo, el orden de nombre y precio. Observa que las tres tarjetas cambian de presentación porque comparten la misma definición. Si copias tres artículos manualmente en la página, ya no estás demostrando reutilización; habrás regresado al estado anterior.

Fase D — Construir interactividad sin duplicar la tarjeta

En P3.4 el padre añade campos de entrada y estado local. Define el campo seleccion, el texto nuevoNombre, el identificador siguienteId y los campos para editar por Id. Sitúa el botón de selección **junto a** la instancia de

TarjetaProducto, dentro del mismo contenedor de iteración; todavía no es necesario enseñar EventCallback. Comprueba que cada botón transmite producto.Id desde su propia iteración y que los botones pertenecen al producto correcto incluso cuando cambias el orden.

Activa @bind y el método de alta. El valor que se escribe en el input se almacena en una propiedad C# del componente; el método Agregar utiliza ese valor para construir un nuevo Producto y lo incorpora a la colección. Para evidenciar la dependencia, introduce Regla: deberías observar un nuevo artículo. Después escribe un nombre compuesto únicamente por espacios. La decisión de no añadirlo proviene del retorno temprano del método, **no** de una regla SQL ni del componente visual.

A continuación activa el editor. SeleccionarParaEditar guarda el Id, no solo el nombre. GuardarCambio localiza por Id y sustituye el record con una expresión with. El nuevo objeto conserva el mismo Id y Precio; no crea un nuevo registro al final. Haz una prueba especialmente reveladora: cambia Lápiz por «Lápiz largo» y comprueba que sigue existiendo un solo elemento con Id 2. Si la cantidad de tarjetas crece, has llamado a Add o has perdido el índice de edición.

Por último ensaya los criterios de consulta. El estado de búsqueda y el precio máximo no deberían sustituir la lista fuente. Construye ProductosVisibles a partir del orden y del filtro; utiliza la propiedad calculada en la iteración de tarjetas. Al escribir un término que no existe, podrás ver un subconjunto vacío sin perder los registros. Al borrar el término, reaparecerán los mismos objetos. Si el editor cambia un nombre, la consulta posterior encontrará ese valor actualizado, porque los datos fuente son compartidos.

Fase E — Reubicar los eventos sin perder capacidades

P3.5 no introduce otra página ni necesita empezar con tres productos distintos. El estado del editor, el método Agregar y los filtros se conservan. Modifica la tarjeta para que pueda emitir AlSeleccionar e incorpora NotificarSeleccion. En el padre retira el botón provisional que estaba junto a la tarjeta y vincula

AlSeleccionar con SeleccionarParaEditar. La nueva ubicación del botón no cambia quién posee el estado.

Completa la segunda intención mediante AlQuitar. El hijo comunica el Id; el padre decide si quita el objeto de la lista y limpia la selección. Enlaza el callback a Quitar, comprueba la eliminación y, cuando comprendas la operación simple, evoluciona el manejo a QuitarConDeshacer. La información necesaria para revertir la última eliminación pertenece al padre, porque solo él conoce la posición de ese objeto en la colección.

Repite una operación antigua tras cada cambio de comunicación. Selecciona un producto para editarlo y comprueba el campo del editor; añade otro y verifica que recibe los botones del hijo; aplica un filtro, elimina un resultado y borra el filtro para comprobar que los demás siguen presentes. Este orden revela si realmente has conservado el proyecto completo o solo has creado otra demostración mínima de EventCallback.

Fase F — Cambiar el origen inicial de los datos, no la aplicación

En P3.6 separa la adquisición de la lista en IProductosMemoriaService y ProductosMemoriaService. El contrato devuelve Task<IReadOnlyList> y la implementación produce datos de laboratorio. Registra el servicio en Program.cs e inyéctalo en la página. La colección de la página puede comenzar vacía y poblarse tras el await: la **responsabilidad del resto de métodos no cambia**. El editor seguirá sustituyendo records, Agregar seguirá construyendo valores y QuitarConDeshacer continuará recordando la última retirada.

Durante la espera, no debemos tratar una lista temporalmente vacía como un resultado definitivo. La bandera cargando permite dibujar el estado provisional y posponer la presentación de controles hasta que el componente haya recibido el resultado. El método OnInitializedAsync se invoca durante la inicialización de la instancia. Una vez completado, el campo estado comunica «Lista preparada» y los datos se representan por TarjetaProducto.

Verifica también la diferencia entre solicitar datos de nuevo y editar datos ya cargados. Una edición local cambia la colección en memoria; llamar al servicio otra vez obtiene una nueva colección inicial. Esta es una frontera real que

habrá que tratar en una aplicación persistente. En este curso no fingimos que un record editado se haya grabado en la base de datos, porque M03 no utiliza almacenamiento externo.

Fase G — Tres resultados posibles de una misma adquisición

Ensaya el resultado correcto con tres productos, el resultado correcto con cero elementos y un fallo controlado. En el segundo caso corresponde un mensaje de vacío, pero no de error. En el tercero la implementación lanza InvalidOperationException de forma deliberada; el componente captura ese caso conocido y presenta «Carga fallida». Si vuelve a intentarlo, debe limpiar el mensaje anterior al comienzo y permitir que la operación llegue de nuevo a un estado correcto.

Durante cada reintento, deshabilita el botón correspondiente. No es necesario programar una segunda interfaz para mostrar el fallo: es otro estado del mismo componente. El usuario desmarca «Simular fallo», pulsa Recargar y recupera el catálogo. Tras esa recuperación selecciona y renombra Lápiz, filtra por su nuevo nombre y elimina/deshace el resultado. Si falla una de estas operaciones, el laboratorio no está completo aunque se haya recuperado la lista.

Fase H — Explicar el programa sin recurrir a frases genéricas

Describe exactamente qué sucede cuando se pulsa Seleccionar dentro de una tarjeta. El botón ejecuta NotificarSeleccion, se invoca AlSeleccionar con el Id de Producto, el padre recibe la intención mediante SeleccionarParaEditar y actualiza tres campos de estado. El framework representa de nuevo la frase de selección y el valor enlazado del editor. La tarjeta no modifica la colección completa y el navegador no recibe código C# para ejecutarlo.

Repite el razonamiento con Guardar nombre: el botón pertenece a Productos.razor, lee nombreEditado, busca el índice por Id, construye otro record y reemplaza el valor de productos. TarjetaProducto vuelve a recibir el objeto actualizado al representarse la lista. El precio y la identidad se conservan porque la expresión with cambia únicamente Nombre.

Finalmente, describe una recarga que falla. La página marca cargando, vacía los datos presentados y llama al contrato del servicio. La implementación decide producir una excepción; el catch conocido transforma esa condición en un mensaje y el finally restablece la disponibilidad del control. Un segundo intento correcto obtiene tres objetos, restablece el estado visible y permite otra vez las operaciones que se aprendieron en P3.4 y P3.5.

El resultado de este taller es un programa cuyo código puede recorrerse desde Program.cs hasta el último botón sin dejar piezas inexplicadas. Si una pieza no aparece en los archivos completos mostrados en las secciones precedentes, hay que volver al paso correspondiente para incorporarla, **no inventarla al margen del manual**.

Autoevaluación integradora

Al final del módulo, abre P3.6 y describe cinco diferencias entre la aplicación MVC A y la aplicación Blazor B sin recurrir a generalidades. Señala el lugar donde se configura Interactive Server; cómo se declara una ruta `@page`; cómo una tarjeta recibe un producto; cómo un método de clic actualiza un texto; y cómo el servicio obtiene datos sin que la página sepa dónde se construye su colección.

Si puedes relacionar estos cinco aspectos con código real, explicar un caso positivo y negativo y distinguir estado de componente de persistencia, habrás alcanzado el objetivo de la jornada. No hay que construir todavía un CRUD nuevo ni desplegar infraestructura adicional.

Taller complementario — Ocho incidentes reproducibles y sus soluciones

Este taller amplía el material de consulta; **no añade ocho actividades obligatorias a las seis horas de clase**. Permite repetir una situación problemática, observar qué falla y recuperarla con un cambio controlado. Cada caso debe realizarse en su carpeta P3.N, partiendo del estado activado que indiquen las instrucciones anteriores. Al terminar, restaura los cambios experimentales. No necesitas otra aplicación, una API, SQL Server ni bibliotecas nuevas.

Incidente A — El método existe, pero el clic no responde (P3.1)

Una persona abre `/estatica`, observa el código `@onclick="Incrementar"` y concluye que el método C# está mal escrito porque el contador no aumenta. Antes de modificar nada, compáralo con `/interactiva`: ambos contienen un botón que invoca un método semejante, pero solo la segunda página declara `@rendermode InteractiveServer`.

Reproducción: ejecuta el proyecto 3.1, abre `/estatica`, pulsa dos veces y comprueba que el valor permanece en cero. Abre `/interactiva`, espera la conexión y pulsa dos veces. Ahora debe aparecer el valor dos. Si ambos botones compilan, su diferencia principal no es aritmética. Comprueba `Program.cs` y `App.razor` para localizar los servicios, endpoints y recursos del host, pero no añadas otro controlador.

Experimento reversible: quita temporalmente la directiva de modo de `Interactiva.razor`, recompila y repite el clic. La página deja de responder como componente interactivo. Restaura la directiva y repite la prueba positiva. El resultado observable identifica una condición concreta del modelo de ejecución.

Explicación causal: una respuesta HTML puede contener el valor inicial de un campo C# sin disponer del mecanismo de eventos que permite modificarlo después del clic. Compilar y descargar la página no sustituyen la prueba de interacción en navegador. La solución no es crear un POST MVC ni añadir JavaScript manualmente: es comprender el modo declarado y su infraestructura.

Incidente B — La página existe, pero no aparece en el menú (P3.2)

Un alumno escribe `/productos` y ve Cuaderno, Lápiz y Carpeta. Después regresa al inicio y no encuentra un enlace Productos. Supone que la ruta está mal registrada. Antes de editar `@page`, observa que la dirección directa ya ha demostrado la existencia del componente.

Reproducción: abre el proyecto 3.2 con `PASO 3.2.2` aún comentado. Visita la ruta directa y después el inicio. Abre `Components/Layout/MainLayout.razor` y activa el

bloque `NavLink href="/productos"`. Compila y comprueba que el enlace aparece y conduce a la misma ruta que antes funcionaba escribiéndola directamente.

Control negativo: modifica temporalmente solo el `href` hacia una ruta inexistente. El menú puede mostrar el enlace, pero su destino no corresponderá a una página. Restablece `/productos` al terminar. Esta prueba separa la existencia visual de un enlace de la existencia del endpoint que representa su destino.

Explicación causal: `@page` declara la ruta; el layout ofrece navegación. Si la ruta funciona pero no tiene enlace, revisa el layout. Si el enlace existe pero el destino muestra contenido de NotFound, revisa las rutas. Si la ruta funciona pero faltan productos, revisa los datos de la página. Cambiar el nombre del record Producto no resuelve ninguno de los dos primeros problemas.

Incidente C — Tres tarjetas, pero las tres contienen Cuaderno (P3.3)

Tras sustituir la lista por TarjetaProducto, el alumno ve tres artículos. Sin embargo, los tres tienen el mismo nombre y referencia. Este escenario distingue un contrato válido de un valor **semánticamente equivocado** transmitido por el padre.

Reproducción: utiliza la carpeta 3.3 con las tarjetas activadas y la lista anterior desactivada. Comprueba primero que cada tarjeta recibe un Id distinto. Cambia temporalmente la expresión `Producto="producto"` del padre por `Producto="productos[0]"`. Compila y observa la pantalla: hay tres instancias de componente, pero todas representan el primer objeto de la colección.

Inspección: localiza en el navegador el atributo `data-producto-id` de cada tarjeta. Si todos contienen el primer identificador, no basta afirmar que el bucle funciona porque se ven tres tarjetas. El bucle se ejecuta, pero transmite un valor equivocado al hijo en cada iteración.

Corrección: restablece `Producto="producto"`. La tarjeta declara una propiedad `[Parameter]` que recibe lo que el padre le proporciona; no puede adivinar qué objeto debería haber recibido. Comprueba de nuevo nombres, precios e

identificadores. La prueba funcional necesita verificar cantidad **e identidad**, no únicamente cantidad de HTML generado.

Incidente D — El botón Añadir no compila después de activarlo (P3.4)

Un alumno descomenta la línea `<button id="agregar" @onclick="Agregar">` y ejecuta `dotnet build`. Aparece un error relativo al método de evento o al campo `nuevoNombre`. Este caso enseña que los pasos numerados tienen dependencias de código, no solo fragmentos visuales.

Preparación: abre un estado inicial limpio de la carpeta 3.4. Localiza tres grupos marcados: `PASO 3.4.2`, `PASO 3.4.3` y `PASO 3.4.4`. Cada uno incluye elementos Razor y miembros C#. Debes activar cada grupo de forma completa antes de comprobarlo, sin copiar métodos de proyectos posteriores.

Secuencia correcta: activa primero el botón Seleccionar y el método `Seleccionar(int)`. Compila y observa su efecto. Activa después el input `@bind="nuevoNombre"` y el campo C# correspondiente. Vuelve a compilar. Finalmente activa el botón Añadir y **todo** su bloque C#, incluido `siguienteId` y `Agregar()`. El método utiliza ambos datos, por lo que no puede compilar correctamente si alguno falta.

Prueba visible: escribe Regla y pulsa Añadir. Debe aparecer una cuarta tarjeta. Prueba una entrada vacía: no se incorpora otro objeto. El resultado no requiere una acción HTTP POST ni un DbContext. Si el botón sigue sin responder pese a compilar, comprueba modo interactivo, enlace de datos y manejador real, distinguiendo sus responsabilidades.

Incidente E — La cuarta tarjeta desaparece después de recargar (P3.4)

En 3.4, un alumno añade Regla y cuenta cuatro artículos. Recarga por completo y vuelve a encontrar solo tres. Lo interpreta como un fallo de persistencia. La comparación con M02 permite demostrar por qué ese criterio no es apropiado para el estado local de esta práctica.

Observación: abre el método `Agregar` y busca una llamada a `productos.Add`. Después busca una operación de almacenamiento persistente, como `SaveChangesAsync` o un servicio que comunique el cambio a una API. El proyecto no

contiene esa infraestructura, porque todavía estamos aprendiendo eventos y estado del componente.

Experimento: selecciona Lápiz, añade Regla y observa que ambos cambios se reflejan. Recarga la página para que se cree una nueva instancia. Los campos vuelven a sus valores iniciales. No es evidencia de que `@onclick` estuviera roto; el manejador ya produjo el cambio requerido durante la sesión.

Explicación causal: en M02 el resultado de un POST podía almacenarse en LocalDB y comprobarse después de reiniciar la aplicación. En P3.4 la lista existe dentro de una instancia interactiva. Su criterio de éxito es la actualización visible en esa instancia, no una promesa de persistencia que no se ha implementado. No utilices una lista estática global para ocultar la diferencia.

Incidente F — La activación de callbacks produce tarjetas duplicadas (P3.5)

La página P3.5 entrega tres variantes de etiqueta `TarjetaProducto`: sin callback, con selección y con selección y eliminación. Si activas una nueva variante pero conservas la línea anterior, cada vuelta del `foreach` representa dos tarjetas, aunque la colección C# siga teniendo tres productos.

Reproducción: sobre una copia del proyecto 3.5, deja activa la variante inicial y activa además la variante con `AlSeleccionar`. Comprueba cuántos artículos HTML aparecen. El error se reproduce sin modificar la lista ni el modelo. No lo corrijas borrando productos del campo `productos`: eso ocultaría la causa y cambiaría los datos que otros pasos esperan.

Corrección: mantén una sola variante activa dentro del `foreach`. Comprueba que el hijo declara los parámetros que esa variante utiliza y que el padre dispone de los métodos receptores. Ahora pulsa Seleccionar sobre Lápiz y verifica el texto del padre. Si continúas con el reto, sustituye otra vez la etiqueta por la variante con `AlQuitar` y comprueba que desaparece un solo producto.

Explicación causal: duplicar instancias de componente es un problema de composición, no una consecuencia automática de EventCallback. El callback

transmite intenciones; el número de tarjetas depende de las etiquetas activas dentro del bucle.

Incidente G — Se quitó la tarjeta, pero continúa seleccionada (P3.5)

El reto de eliminación produce tres cambios relacionados: el producto desaparece, el total disminuye y el texto de selección vuelve a «ninguno». Si falta el último, el callback puede haber funcionado aunque el estado de la interfaz quede incoherente.

Experimento: activa el reto y selecciona Lápiz. En `Quitar(int id)` comenta temporalmente solo `seleccion = "ninguno"`. Pulsa Quitar. La tarjeta puede desaparecer y el total reducirse, mientras la cabecera sigue mencionando el producto eliminado. El problema no está necesariamente en `AlQuitar.InvokeAsync`: el padre recibió el Id y eliminó el objeto.

Corrección: restaura la asignación que limpia la selección y repite. El estado visible debe resultar coherente. Esta prueba enseña a separar **comunicación correcta entre componentes** de **reglas correctas de actualización del estado**. El componente hijo no debería acceder por su cuenta al campo de selección del padre; simplemente comunica la intención de quitar un producto.

Transferencia: en una aplicación mayor, el padre podría solicitar confirmación antes de quitar. El hijo continuaría enviando un Id. No es necesario incorporar esa funcionalidad en M03 para apreciar la ventaja de un contrato de callback pequeño y explícito.

Incidente H — El servicio devuelve productos, pero no se dibujan (P3.6)

El componente de P3.6 obtiene tres objetos mediante `Catalogo.ObtenerAsync()` y los asigna a `productos`. Sin embargo, el marcado requiere además que `cargando` sea falso. Es posible que existan datos correctos y que la representación todavía no esté habilitada por el propio estado de presentación.

Reproducción: activa `OnInitializedAsync` y comprueba la carga normal. Después comenta temporalmente `cargando = false`. El servicio puede completar su tarea, pero el bloque `@if (!cargando && productos.Count > 0)` no representa tarjetas. No se

trata de un fallo de CSS ni de una API remota: la condición está esperando una bandera que nunca cambia.

Corrección: restaura la asignación y comprueba que las tarjetas aparecen. Activa ahora el reto de lista vacía y cambia el retorno del servicio por `Array.Empty<Producto>()`. El resultado correcto es distinto: la carga termina y la pantalla muestra «No hay productos para mostrar» porque no existen elementos. No debe confundirse con esperar indefinidamente.

Explicación causal: la secuencia completa es registro del servicio, inyección, inicialización, espera asíncrona, asignación de la lista, actualización de `cargando` y representación. La ausencia de tarjetas puede indicar fallos diferentes según cuál de esos pasos no se haya completado. En M04 añadiremos fallos HTTP, pero todavía no son una causa posible del servicio local de P3.6.

Recuperación del estado de trabajo

Tras cada experimento, revierte únicamente el cambio utilizado para provocarlo. Si tienes dudas, recupera el archivo de la carpeta de la práctica que estás realizando, no el de otro punto. El anexo de archivos canónicos de este mismo cuaderno permite comparar el código con el estado inicial versionado.

Al explicar un incidente, escribe tres elementos: **síntoma observado**, **hipótesis sobre la responsabilidad que falla** y **prueba que distingue esa hipótesis de otras**. Esta secuencia ayuda más que modificar varios archivos a la vez. El objetivo es que el alumno pueda razonar sobre su propia aplicación y no dependa de reconstruir la solución completa cada vez que un botón o una página no se comporta como espera.

Referencia de código para el recorrido acumulativo

Este manual presenta los archivos completos en el punto donde se estudia cada cambio: P3.1 incluye el arranque y las páginas comparativas; P3.2 añade modelo, catálogo y navegación; P3.3 introduce la tarjeta y la página compuesta; P3.4 desarrolla estado, eventos, alta, edición y consulta; P3.5 incorpora callbacks y recuperación sin perder funciones; P3.6 obtiene

inicialmente los mismos objetos mediante un servicio asíncrono y conserva toda la interacción.

Las instantáneas 3.1–3.6 permiten reproducir cada etapa sobre la misma aplicación B sin tener que editar los proyectos de módulos anteriores. Los archivos compartidos del arranque, estilos y navegación siguen siendo los mismos conceptos: se explica su código una sola vez y únicamente se explica de nuevo cuando cambia su responsabilidad. Si el alumno ve una función de las prácticas que no encuentra en el archivo de su instantánea, debe comprobar que está en el punto correcto y que ha activado los comentarios docentes correspondientes.

Fuentes de consulta

- Microsoft Learn: Fundamentos de Blazor.
- Microsoft Learn: Modos de renderizado.
- Microsoft Learn: Componentes, parámetros y composición.
- Microsoft Learn: Eventos y EventCallback.
- Microsoft Learn: Ciclo de vida de los componentes.